

Coding and Cryptography

Adam Kelly (ak2316@cam.ac.uk)

July 27, 2026

Communication is a mathematical problem. Somebody wants to say something to somebody else; between them lies a channel that is expensive, or unreliable, or overheard, or all three at once. Coding theory is the study of how to package a message so that it survives the journey, and cryptography is the study of how to package it so that only the intended recipient can read it. Both subjects turn out to be, at bottom, about the same thing: the arithmetic of finite sets, and the surprising leverage that a little algebra and a little probability give us over them.

The most striking result in the course is due to Shannon, and it says something that ought to be false. If a channel corrupts one bit in ten, entirely at random, one might reasonably guess that transmitting a long message without error requires sending it over and over again, so that the rate at which information gets through must fall to zero. It does not. There is a positive number – the *capacity* of the channel – below which arbitrarily reliable communication is possible, and Shannon’s theorem tells us exactly what that number is. What makes the proof so memorable is that it establishes the existence of extremely good codes by choosing one at random, without ever exhibiting a single one.

This article constitutes my notes for the ‘Coding and Cryptography’ course, held in Lent 2021 at Cambridge. These notes are *not a transcription of the lectures*, and differ significantly in quite a few areas. Still, all lectured material should be covered.

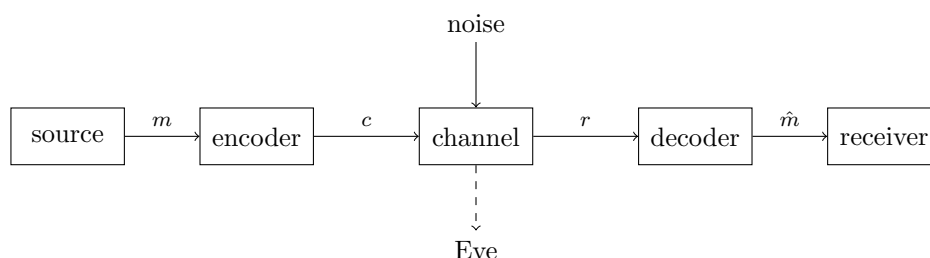
Contents

1	Modelling Communication	2
2	Noiseless Coding	5
2.1	Codes and Decipherability	5
2.2	Kraft’s Inequality	6
2.3	McMillan’s Inequality	7
2.4	Entropy	8
2.5	Optimal Codes and the Noiseless Coding Theorem	10
2.6	Huffman Coding	12
2.7	Joint Entropy	14
3	Noisy Channels and Error-Correcting Codes	15
3.1	Decoding Rules	15
3.2	Error Detection and Correction	17
3.3	Covering Estimates and Perfect Codes	19
3.4	Asymptotics	22

3.5	Constructing New Codes from Old	24
4	Information Theory	25
4.1	Sources and Information Rate	25
4.2	The Asymptotic Equipartition Property	27
4.3	Shannon's First Coding Theorem	28
4.4	Capacity of the Binary Symmetric Channel	29
4.5	Conditional Entropy and Fano's Inequality	30
4.6	Mutual Information and Information Capacity	32
4.7	Proof of the Noisy Coding Theorem	34
4.8	An Aside: the Kelly Criterion	36

§1 Modelling Communication

Before we can prove anything we need a model, and the one we shall use is the one Shannon drew in 1948. There is a **source** which produces a message; an **encoder** which turns that message into something suitable for transmission; a **channel** down which the transmission travels and in which things go wrong; a **decoder** which tries to undo the damage; and finally a **receiver**, for whom the whole business was arranged.



By long-standing convention the source is called Alice, the receiver is called Bob, and any third party listening in on the channel is called Eve. The two halves of the course correspond to the two things that can go wrong in the middle of that diagram. In coding theory the enemy is the noise, and we want the message to arrive *economically* and *reliably*. In cryptography the enemy is Eve, and we want the message to arrive *privately*.

It is worth separating two quite different problems which are often confused. **Noiseless coding**, or source coding, is about compression: we want to spend as few symbols as possible on a message, and it is adapted to the *source*. Morse code is the canonical example – the letter E, the commonest letter in English, gets the single dot, and the letter Q gets four symbols. **Noisy coding**, or channel coding, is about redundancy: we want to add enough extra symbols that corruption can be spotted and undone, and it is adapted to the *channel*.

Example 1.1 (ISBNs)

Every book (before 2007) carried a ten-digit International Standard Book Number $a_1a_2\dots a_{10}$, where $a_1, \dots, a_9 \in \{0, 1, \dots, 9\}$ and $a_{10} \in \{0, 1, \dots, 9, X\}$, subject to

the single condition

$$\sum_{j=1}^{10} ja_j \equiv 0 \pmod{11}.$$

Here X stands for 10, and the modulus 11 is prime, which is what makes everything work. Suppose one digit is mistyped, say a_k becomes a'_k . Then the check sum changes by $k(a_k - a'_k)$, and since $1 \leq k \leq 10$ and $|a_k - a'_k| \leq 10$, neither factor is divisible by 11, so the sum is no longer 0 modulo 11 and the error is detected. Similarly, if two adjacent digits are transposed, say a_k and a_{k+1} swap, the sum changes by $a_{k+1} - a_k$, again nonzero modulo 11 unless the digits were equal in the first place.

The channel here is a human being reading a number off a page and typing it into a computer, and the code is beautifully adapted to the errors that channel makes.

We now make the notion of a channel precise.

Definition 1.2 (Communication Channel)

A **communication channel** accepts strings of symbols from a finite **input alphabet** $\mathcal{A} = \{a_1, \dots, a_r\}$ and outputs strings of symbols from a finite **output alphabet** $\mathcal{B} = \{b_1, \dots, b_s\}$. It is specified by the probabilities

$$\mathbb{P}(y_1 \dots y_n \text{ received} \mid x_1 \dots x_n \text{ sent}).$$

In this generality nothing can be said, because the channel is allowed to remember everything it has ever done. Almost always we assume much more.

Definition 1.3 (Discrete Memoryless Channel)

A **discrete memoryless channel** (DMC) is a channel for which the probabilities

$$p_{ij} = \mathbb{P}(b_j \text{ received} \mid a_i \text{ sent})$$

are the same on every use of the channel, and are independent of all past and future uses. The $r \times s$ matrix $P = (p_{ij})$ is the **channel matrix**; each of its rows sums to 1.

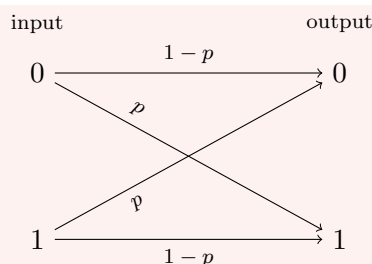
Two examples will accompany us for the rest of the course.

Example 1.4 (Binary Symmetric Channel)

The **binary symmetric channel** (BSC) with error probability $p \in [0, 1]$ has $\mathcal{A} = \mathcal{B} = \{0, 1\}$ and channel matrix

$$P = \begin{pmatrix} 1-p & p \\ p & 1-p \end{pmatrix}.$$

Each bit is flipped independently with probability p , and transmitted faithfully with probability $1 - p$.



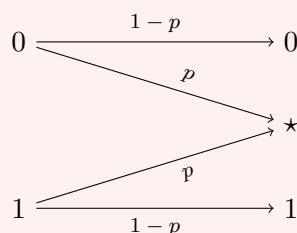
We shall almost always assume $p < \frac{1}{2}$. This costs nothing: a channel with $p > \frac{1}{2}$ becomes one with error probability $1-p$ if Bob simply inverts everything he receives, and we shall see shortly that a channel with $p = \frac{1}{2}$ is useless, in that it carries no information whatsoever.

Example 1.5 (Binary Erasure Channel)

The **binary erasure channel** (BEC) has $\mathcal{A} = \{0, 1\}$ and $\mathcal{B} = \{0, 1, \star\}$, with channel matrix

$$P = \begin{pmatrix} 1-p & 0 & p \\ 0 & 1-p & p \end{pmatrix}.$$

Here p is the probability that the symbol arrives illegibly; receiving $\star$ is called a **splurge error**. This channel never lies to us, it merely occasionally declines to speak.



To use a channel n times we pass to the **n th extension**, whose input alphabet is $\mathcal{A}^n$, whose output alphabet is $\mathcal{B}^n$, and whose channel matrix (for a DMC) is

$$\mathbb{P}(y_1 \dots y_n \mid x_1 \dots x_n) = \prod_{i=1}^n \mathbb{P}(y_i \mid x_i).$$

Definition 1.6 (Code, Rate, Error Rate)

A **code** C of length n is a function $\mathcal{M} \rightarrow \mathcal{A}^n$, where $\mathcal{M}$ is a set of messages; implicitly we also fix a decoding rule $\mathcal{B}^n \rightarrow \mathcal{M}$. We usually identify C with its image, a subset of $\mathcal{A}^n$.

- The **size** of C is $m = |\mathcal{M}|$.
- The **information rate** of C is $\rho(C) = \frac{1}{n} \log_2 m$.
- The **error rate** of C is $\hat{e}(C) = \max_{x \in \mathcal{M}} \mathbb{P}(\text{error} \mid x \text{ sent})$.

Throughout these notes $\log$ means $\log_2$ unless we say otherwise; this is the convention that makes the answers come out in bits.

Definition 1.7 (Capacity)

A channel can **transmit reliably at rate** R if there is a sequence of codes $(C_n)_{n \geq 1}$, with C_n of length n , such that

$$\lim_{n \rightarrow \infty} \rho(C_n) = R \quad \text{and} \quad \lim_{n \rightarrow \infty} \hat{e}(C_n) = 0.$$

The **capacity** of the channel is the supremum of all rates at which it can transmit reliably.

Note carefully what is being asked. We are not asking for one good code; we are asking for a whole family of codes whose rate stays bounded away from zero while the error probability is driven to zero. It is far from obvious that this can be done at all for a channel that makes errors, and the fact that it can – that the binary symmetric channel with $p < \frac{1}{2}$ has strictly positive capacity – is Shannon’s noisy coding theorem, which we prove in Section 4.7.

§2 Noiseless Coding

We begin with the easier half of the problem, in which the channel is perfect and the only question is economy. A source emits symbols from an alphabet $\mathcal{A}$, some more often than others, and we want to represent them as short strings over a transmission alphabet $\mathcal{B}$.

§2.1 Codes and Decipherability

Let $\mathcal{B}$ be a finite alphabet. Write $\mathcal{B}^* = \bigcup_{n \geq 0} \mathcal{B}^n$ for the set of all finite strings over $\mathcal{B}$, including the empty string. The **concatenation** of $x = x_1 \dots x_r$ and $y = y_1 \dots y_s$ is $xy = x_1 \dots x_r y_1 \dots y_s$.

Definition 2.1 (Code)

Let $\mathcal{A}$ and $\mathcal{B}$ be alphabets. A **code** is a function $c: \mathcal{A} \rightarrow \mathcal{B}^*$. The elements of the image of c are the **codewords**.

We write $m = |\mathcal{A}|$ and $a = |\mathcal{B}|$, and call c an **a -ary code of size m** . A 2-ary code is *binary*, a 3-ary code is *ternary*.

Example 2.2 (The Greek Fire Code)

Take $\mathcal{A} = \{\alpha, \beta, \dots, \omega\}$, the 24 letters of the Greek alphabet, and $\mathcal{B} = \{1, 2, 3, 4, 5\}$. Arrange the letters in a 5×5 square and encode each by its coordinates, so $c(\alpha) = 11$, $c(\beta) = 12$, and so on. To transmit the pair xy one holds up x torches on one hilltop and y torches on another. This scheme is described by Polybius, and is very probably the oldest code in these notes by some two thousand years.

A code is only useful if we can send more than one symbol. Given a message $x_1 x_2 \dots x_n \in \mathcal{A}^*$ we transmit $c(x_1)c(x_2) \dots c(x_n)$, so that c extends to a map

$$c^*: \mathcal{A}^* \rightarrow \mathcal{B}^*.$$

Definition 2.3 (Decipherable)

A code c is **decipherable**, or **uniquely decodable**, if c^* is injective.

It is not enough for c itself to be injective, as the following shows.

Example 2.4

Let $\mathcal{A} = \{1, 2, 3, 4\}$ and $\mathcal{B} = \{0, 1\}$, and set

$$c(1) = 0, \quad c(2) = 1, \quad c(3) = 00, \quad c(4) = 01.$$

This is injective, but $c^*(114) = 0001 = 0001$ and $c^*(312) = 0001 = 0001$, so c^* is not. On receiving 0001 Bob has no way to tell which message was meant.

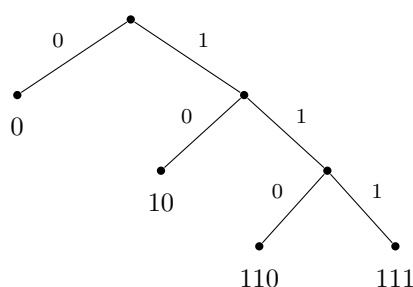
There are three easy ways of guaranteeing decipherability, assuming c injective.

- (i) A **block code**, in which all codewords have the same length. The Greek fire code is of this kind.
- (ii) A **comma code**, which reserves one letter of $\mathcal{B}$ to mark the end of a codeword and never uses it otherwise.
- (iii) A **prefix-free code**, in which no codeword is an initial segment of another.

Block codes and comma codes are both special cases of prefix-free codes. Prefix-free codes have the pleasant property that Bob can decode as he reads: the moment the symbols he has seen form a codeword, he knows that codeword has ended, because no longer codeword begins that way. For this reason they are also called **instantaneous codes**.

Not every decipherable code is prefix-free. For instance $c(1) = 0$, $c(2) = 01$, $c(3) = 011$, $c(4) = 0111$ is decipherable – each 0 marks the start of a new codeword – but $c(1)$ is a prefix of $c(2)$. Here Bob must look ahead to the next 0 before committing. Remarkably, we shall see in a moment that allowing such lookahead buys us precisely nothing.

The right way to picture a prefix-free binary code is as a rooted binary tree. Reading a codeword left to right tells us which way to turn at each vertex, and the prefix-free condition says exactly that all the codewords sit at *leaves*.

**§2.2 Kraft's Inequality**

How short can the codewords of a prefix-free code be? Not arbitrarily short: the tree picture makes it clear that a short codeword uses up a lot of the tree. The following inequality measures exactly how much.

Definition 2.5 (Kraft's Inequality)

Let $c: \mathcal{A} \rightarrow \mathcal{B}^*$ be a code with $|\mathcal{A}| = m$ and $|\mathcal{B}| = a$, whose codewords have lengths $\ell_1, \dots, \ell_m$. We say that c satisfies **Kraft's inequality** if

$$\sum_{i=1}^m a^{-\ell_i} \leq 1.$$

Theorem 2.6 (Kraft)

Given lengths $\ell_1, \dots, \ell_m$, there is a prefix-free a -ary code with these codeword lengths if and only if Kraft's inequality holds.

Proof. Let $s = \max_i \ell_i$ and let n_ℓ be the number of i with $\ell_i = \ell$, so that Kraft's inequality reads $\sum_{\ell=1}^s n_\ell a^{-\ell} \leq 1$, or after multiplying by a^s ,

$$n_1 a^{s-1} + n_2 a^{s-2} + \dots + n_{s-1} a + n_s \leq a^s. \quad (*)$$

Suppose first that c is prefix-free. A codeword of length ℓ is a prefix of exactly $a^{s-\ell}$ strings of length s , and by the prefix-free condition no string of length s has two different codewords as prefixes. So the left hand side of $(*)$ counts, without repetition, some of the strings of length s , of which there are a^s in all. Hence $(*)$ holds.

Conversely suppose $(*)$ holds; we build a prefix-free code with n_ℓ codewords of length ℓ for each ℓ , by induction on s . If $s = 1$ the inequality says $n_1 \leq a$ and we simply choose n_1 of the a letters. For $s > 1$, the inductive hypothesis applied to the lengths at most $s - 1$ gives a prefix-free code $\hat{c}$ with n_ℓ codewords of length ℓ for every $\ell < s$. Now the strings of length s having some codeword of $\hat{c}$ as a prefix number exactly

$$n_1 a^{s-1} + \dots + n_{s-1} a,$$

so by $(*)$ at least n_s strings of length s remain untouched. Choosing n_s of them and adjoining them to $\hat{c}$ gives a code which is still prefix-free, since none of the new codewords extends an old one (they were chosen to avoid this) and none of the old codewords extends a new one (they are shorter). $\square$

Remark. The proof is constructive, and gives a perfectly practical algorithm: list the required lengths in increasing order and greedily assign codewords, at each stage taking any string of the right length that does not extend a codeword already chosen. Kraft's inequality guarantees that we never run out.

§2.3 McMillan's Inequality

Now we come to the result promised above, which says that lookahead is worthless.

Theorem 2.7 (McMillan)

Every decipherable code satisfies Kraft's inequality.

Proof. Let $c: \mathcal{A} \rightarrow \mathcal{B}^*$ be decipherable with word lengths $\ell_1, \dots, \ell_m$, and set $s = \max_i \ell_i$. The trick is to consider the R th power of the quantity in question. For any $R \in \mathbb{N}$,

$$\left(\sum_{i=1}^m a^{-\ell_i} \right)^R = \sum_{\ell=R}^{Rs} b_\ell a^{-\ell},$$

where b_ℓ is the number of ways of choosing an ordered R -tuple of codewords whose total length is ℓ ; this is just what happens when we expand the product. Now a tuple of R codewords of total length ℓ concatenates to a string in $\mathcal{B}^\ell$, and because c is decipherable, distinct tuples give distinct strings. Hence

$$b_\ell \leq |\mathcal{B}^\ell| = a^\ell,$$

and so each term $b_\ell a^{-\ell}$ is at most 1. There are at most Rs terms, so

$$\left(\sum_{i=1}^m a^{-\ell_i} \right)^R \leq Rs, \quad \text{that is} \quad \sum_{i=1}^m a^{-\ell_i} \leq (Rs)^{1/R}.$$

Letting $R \rightarrow \infty$, the right hand side tends to 1, and Kraft's inequality follows. $\square$

The proof is a small piece of magic: an inequality that fails to hold is amplified by exponentiation into an absurdity, while the polynomial factor Rs is powerless to keep up.

Corollary 2.8

A decipherable code with word lengths $\ell_1, \dots, \ell_m$ exists if and only if a prefix-free code with the same word lengths exists.

Proof. A prefix-free code is decipherable. Conversely, a decipherable code satisfies Kraft's inequality by McMillan, and then Kraft's theorem produces a prefix-free code with the same lengths. $\square$

So from the point of view of word lengths – which is the only point of view that matters for economy – we may as well restrict attention to prefix-free codes, and we shall.

§2.4 Entropy

We now need a way of saying how much information a source actually produces, so that we know what we are aiming at. The right notion is Shannon's **entropy**. Informally, the entropy of a random variable X is the expected number of fair coin tosses needed to determine its value.

Example 2.9

Suppose X takes four values with probability $\frac{1}{4}$ each. Identifying the values with 00, 01, 10, 11, two coin tosses always suffice and never fewer, so the entropy should be 2.

Suppose instead X takes four values with probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$. Identify them with the strings 0, 10, 110, 111: toss a coin, and if it comes up 0 we are done, otherwise

toss again, and so on. The expected number of tosses is

$$\frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = \frac{7}{4}.$$

The second variable is in this sense less random than the first.

Both examples are instances of the following.

Definition 2.10 (Entropy)

Let X be a random variable taking finitely many values $x_1, \dots, x_n$ with probabilities $p_1, \dots, p_n$. The **entropy** of X is

$$H(X) = H(p_1, \dots, p_n) = - \sum_{i=1}^n p_i \log p_i = \mathbb{E}[-\log p(X)],$$

where $\log = \log_2$ and we adopt the convention $0 \log 0 = 0$.

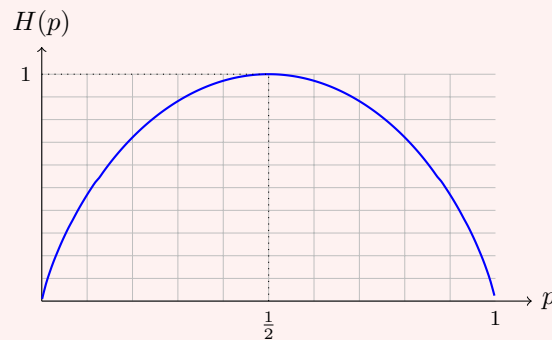
The convention is forced on us by continuity, since $x \log x \rightarrow 0$ as $x \rightarrow 0^+$. Entropy is measured in **bits**. Clearly $H(X) \geq 0$, with equality if and only if some $p_i = 1$, that is, if and only if X is constant.

Example 2.11 (The Binary Entropy Function)

For a biased coin with probability p of heads we write

$$H(p) = H(p, 1-p) = -p \log p - (1-p) \log(1-p).$$

Differentiating, $H'(p) = \log \frac{1-p}{p}$, which is positive for $p < \frac{1}{2}$ and negative for $p > \frac{1}{2}$; and $H''(p) < 0$ throughout, so H is strictly concave with a maximum of 1 at $p = \frac{1}{2}$, vanishing at $p = 0$ and $p = 1$.



This little function is going to appear everywhere, so it is worth committing its shape to memory: symmetric about $\frac{1}{2}$, with infinite slope at the two ends.

The following inequality is the workhorse of the whole subject.

Proposition 2.12 (Gibbs' Inequality)

Let $(p_1, \dots, p_n)$ and $(q_1, \dots, q_n)$ be probability distributions. Then

$$-\sum_{i=1}^n p_i \log p_i \leq -\sum_{i=1}^n p_i \log q_i,$$

with equality if and only if $p_i = q_i$ for all i .

The right hand side is the **cross entropy** of q relative to p ; the difference between the two sides is called the relative entropy, or Kullback–Leibler divergence, and Gibbs’ inequality says it is non-negative.

Proof. Since $\log x = \ln x / \ln 2$ we may work with natural logarithms throughout. Let $I = \{i : p_i \neq 0\}$. The elementary inequality $\ln x \leq x - 1$ for $x > 0$, with equality only at $x = 1$, gives for $i \in I$

$$\ln \frac{q_i}{p_i} \leq \frac{q_i}{p_i} - 1,$$

and multiplying by $p_i > 0$ and summing over I ,

$$\sum_{i \in I} p_i \ln \frac{q_i}{p_i} \leq \sum_{i \in I} q_i - \sum_{i \in I} p_i \leq 1 - 1 = 0,$$

using $\sum_{i \in I} p_i = 1$ and $\sum_{i \in I} q_i \leq 1$. Therefore

$$-\sum_{i=1}^n p_i \ln p_i = -\sum_{i \in I} p_i \ln p_i \leq -\sum_{i \in I} p_i \ln q_i \leq -\sum_{i=1}^n p_i \ln q_i,$$

the last step because the omitted terms have $p_i = 0$. If equality holds throughout we need $\sum_{i \in I} q_i = 1$ and $q_i/p_i = 1$ for every $i \in I$; the first forces $q_i = 0$ for $i \notin I$, and the second gives $p_i = q_i$ on I . $\square$

Corollary 2.13

$H(p_1, \dots, p_n) \leq \log n$, with equality if and only if $p_1 = \dots = p_n = \frac{1}{n}$.

Proof. Take $q_i = \frac{1}{n}$ in Gibbs’ inequality: the right hand side is $-\sum p_i \log \frac{1}{n} = \log n$. $\square$

In other words, the uniform distribution is the most random one, which is what one would hope.

§2.5 Optimal Codes and the Noiseless Coding Theorem

Fix an alphabet $\mathcal{A} = \{\mu_1, \dots, \mu_m\}$ of $m \geq 2$ messages and a transmission alphabet $\mathcal{B}$ with $a = |\mathcal{B}| \geq 2$. Let X be a random variable on $\mathcal{A}$ with $\mathbb{P}(X = \mu_i) = p_i$. A code c with word lengths $\ell_1, \dots, \ell_m$ has expected word length

$$\mathbb{E}[S] = \sum_{i=1}^m p_i \ell_i.$$

Definition 2.14 (Optimal Code)

A code $c: \mathcal{A} \rightarrow \mathcal{B}^*$ is **optimal** if it has the smallest expected word length among all decipherable codes for X .

We can now say precisely how well one can do.

Theorem 2.15 (Shannon's Noiseless Coding Theorem)

The expected word length $\mathbb{E}[S]$ of an optimal code satisfies

$$\frac{H(X)}{\log a} \leq \mathbb{E}[S] < \frac{H(X)}{\log a} + 1.$$

Moreover the lower bound holds for *every* decipherable code, and is attained exactly when $p_i = a^{-\ell_i}$ for integers ℓ_i (so that in particular $\sum_i a^{-\ell_i} = 1$).

Proof. The lower bound. Let c be any decipherable code with word lengths $\ell_1, \dots, \ell_m$. Put $D = \sum_i a^{-\ell_i}$ and $q_i = a^{-\ell_i}/D$, so that (q_i) is a probability distribution. By Gibbs' inequality,

$$H(X) \leq - \sum_i p_i \log q_i = - \sum_i p_i (-\ell_i \log a - \log D) = \log D + \log a \sum_i p_i \ell_i.$$

By McMillan's inequality $D \leq 1$, so $\log D \leq 0$, and therefore

$$H(X) \leq \log a \cdot \mathbb{E}[S],$$

which is the lower bound. Equality forces $D = 1$ and $p_i = q_i = a^{-\ell_i}$.

The upper bound. It suffices to exhibit *some* decipherable code beating the bound, since an optimal code can only be better. Set

$$\ell_i = \lceil -\log_a p_i \rceil,$$

so that $-\log_a p_i \leq \ell_i < -\log_a p_i + 1$. From the first inequality $p_i \geq a^{-\ell_i}$, and summing, $\sum_i a^{-\ell_i} \leq 1$. So Kraft's inequality holds and there is a prefix-free code c with these word lengths. From the second inequality,

$$\mathbb{E}[S] = \sum_i p_i \ell_i < \sum_i p_i (-\log_a p_i + 1) = \frac{H(X)}{\log a} + 1. \quad \square$$

The code constructed in the second half is worth a name.

Definition 2.16 (Shannon–Fano Coding)

The **Shannon–Fano code** for probabilities $p_1, \dots, p_m$ takes word lengths $\ell_i = \lceil -\log_a p_i \rceil$ and then assigns codewords greedily in order of increasing length, as in the proof of Kraft's theorem.

Example 2.17

Take $a = 2$ and the source

i	1	2	3	4	5	6
p_i	0.30	0.22	0.18	0.14	0.10	0.06

The entropy is

$$H(X) = - \sum_i p_i \log p_i \approx 2.4198.$$

The Shannon–Fano lengths are $\ell_i = \lceil -\log_2 p_i \rceil$, namely

$$\ell = (2, 3, 3, 3, 4, 5),$$

and $\sum_i 2^{-\ell_i} = \frac{1}{4} + 3 \cdot \frac{1}{8} + \frac{1}{16} + \frac{1}{32} = 0.71875 \leq 1$ as it must be. Assigning codewords greedily gives

i	1	2	3	4	5	6
$c(i)$	00	010	011	100	1010	10110

with expected word length

$$\mathbb{E}[S] = 0.30 \cdot 2 + 0.22 \cdot 3 + 0.18 \cdot 3 + 0.14 \cdot 3 + 0.10 \cdot 4 + 0.06 \cdot 5 = 2.92.$$

This does obey $H(X) \leq \mathbb{E}[S] < H(X) + 1$, but only just, and it is visibly wasteful: nothing at all uses the string 11, and we could shorten $c(6)$ to 1011 without breaking anything. Shannon–Fano coding is not optimal.

§2.6 Huffman Coding

Huffman’s algorithm produces a genuinely optimal code, and it does so by a beautifully simple greedy rule. We take $a = 2$ and $\mathcal{B} = \{0, 1\}$ for simplicity; everything generalises.

Order the messages so that $p_1 \geq p_2 \geq \dots \geq p_m$. We define the code recursively.

Definition 2.18 (Huffman Code)

If $m = 2$, take the codewords 0 and 1. If $m > 2$, form the source on $m - 1$ symbols $\mu_1, \dots, \mu_{m-2}, \nu$ with probabilities $p_1, \dots, p_{m-2}, p_{m-1} + p_m$, take a **Huffman code** for it, and then set

$$c(\mu_{m-1}) = c(\nu)0, \quad c(\mu_m) = c(\nu)1,$$

keeping all other codewords unchanged.

In other words: repeatedly merge the two least likely symbols into one, until only two remain; then read the resulting binary tree from the root outwards. Huffman codes are prefix-free by construction, since every codeword sits at a leaf. They are not unique – ties in the probabilities force arbitrary choices – but we shall see that all the choices are equally good.

Example 2.19

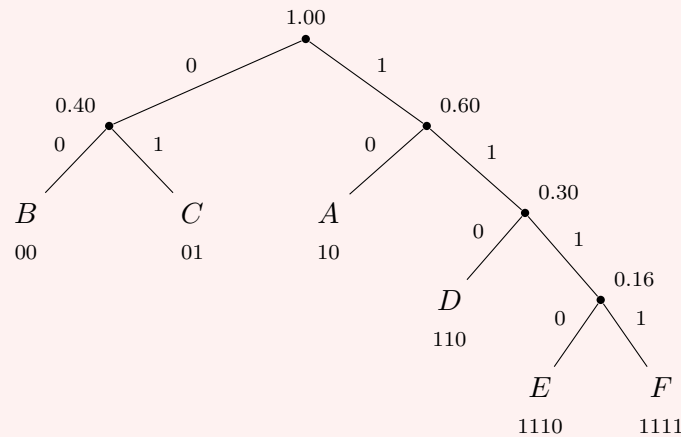
Take the same source as Example 2.17, writing the messages as $A, \dots, F$:

	A	B	C	D	E	F
p	0.30	0.22	0.18	0.14	0.10	0.06

We merge repeatedly.

- The two smallest are E and F ; merge them into X with probability 0.16. Remaining: A 0.30, B 0.22, C 0.18, X 0.16, D 0.14.
- The two smallest are D and X ; merge them into Y with probability 0.30. Remaining: A 0.30, Y 0.30, B 0.22, C 0.18.
- The two smallest are C and B ; merge them into Z with probability 0.40. Remaining: A 0.30, Y 0.30, Z 0.40.
- The two smallest are A and Y ; merge them into W with probability 0.60.
- Finally merge Z and W .

The resulting tree is as follows, with 0 on the left branch and 1 on the right.



Reading off, the word lengths are 2, 2, 2, 3, 4, 4 and

$$\mathbb{E}[S] = 0.30 \cdot 2 + 0.22 \cdot 2 + 0.18 \cdot 2 + 0.14 \cdot 3 + 0.10 \cdot 4 + 0.06 \cdot 4 = 2.46,$$

comfortably better than the 2.92 of Shannon–Fano, and within 0.05 of the entropy bound 2.4198.

To prove optimality we need two structural facts about optimal codes, both of which are obvious once stated.

Lemma 2.20

Let c be an optimal prefix-free code for messages $\mu_1, \dots, \mu_m$ with probabilities $p_1 \geq \dots \geq p_m$, and word lengths $\ell_1, \dots, \ell_m$. Then

- (i) if $p_i > p_j$ then $\ell_i \leq \ell_j$;
- (ii) among the codewords of maximal length, there are two which differ only in their last digit.

Proof. (i). Suppose $p_i > p_j$ but $\ell_i > \ell_j$. Swapping the i th and j th codewords leaves the code prefix-free and changes the expected word length by

$$(p_i \ell_j + p_j \ell_i) - (p_i \ell_i + p_j \ell_j) = (p_i - p_j)(\ell_j - \ell_i) < 0,$$

contradicting optimality.

(ii). Let $\ell = \max_i \ell_i$, and suppose no two codewords of length ℓ agree in their first $\ell - 1$ digits. Deleting the final digit of every codeword of length ℓ then leaves all these codewords distinct; and the code is still prefix-free, because a shortened codeword y could only be a prefix of some other codeword z if $y0$ or $y1$ were, and z has length at most ℓ , so $z \in \{y0, y1\}$, which our assumption excludes. But this strictly decreases the expected word length, contradicting optimality. $\square$

Theorem 2.21

Huffman codes are optimal.

Proof. We induct on m . For $m = 2$ the Huffman code has both word lengths equal to 1, which is clearly best possible.

Let $m > 2$ and let c_m be the Huffman code for X_m taking values $\mu_1, \dots, \mu_m$ with probabilities $p_1 \geq \dots \geq p_m$. By construction c_m is built from a Huffman code c_{m-1} for the merged source X_{m-1} on $\mu_1, \dots, \mu_{m-2}, \nu$ with probabilities $p_1, \dots, p_{m-2}, p_{m-1} + p_m$, by extending the codeword of ν by one digit in each of two ways. Comparing expected lengths term by term,

$$\mathbb{E}[S_m] = \mathbb{E}[S_{m-1}] + p_{m-1} + p_m. \quad (1)$$

Now let c'_m be an optimal code for X_m ; by the corollary to McMillan's theorem we may take it prefix-free. By Lemma 2.20(i) the two longest codewords belong to two of the least likely messages, and by (ii) we may assume, after relabelling messages of equal probability and swapping codewords of equal length, that

$$c'_m(\mu_{m-1}) = y0, \quad c'_m(\mu_m) = y1$$

for some string y . Define a code c'_{m-1} for X_{m-1} by

$$c'_{m-1}(\mu_i) = c'_m(\mu_i) \quad (i \leq m-2), \quad c'_{m-1}(\nu) = y.$$

This is prefix-free: no codeword of c'_m other than $y0, y1$ can have y as a prefix, else it would have $y0$ or $y1$ as a prefix. Exactly as in (1),

$$\mathbb{E}[S'_m] = \mathbb{E}[S'_{m-1}] + p_{m-1} + p_m. \quad (2)$$

By the inductive hypothesis c_{m-1} is optimal for X_{m-1} , so $\mathbb{E}[S_{m-1}] \leq \mathbb{E}[S'_{m-1}]$. Combining with (1) and (2),

$$\mathbb{E}[S_m] \leq \mathbb{E}[S'_m],$$

and since c'_m was optimal, so is c_m . $\square$

Remark. Not every optimal code is a Huffman code – one can always relabel $0 \leftrightarrow 1$

in a subtree, for instance. What the theorem really gives us is that the Huffman word lengths are optimal word lengths.

Huffman coding is optimal but not perfect: it can never do better than the entropy bound, and if one symbol has probability close to 1 it must still spend a whole bit on it. The remedy is to Huffman-code blocks of N symbols at a time, which as we shall see in Section 4 drives the cost per symbol down to the entropy.

§2.7 Joint Entropy

Finally in this section we record how entropy behaves for pairs of random variables, which we shall need repeatedly later on.

If X, Y take values in $\mathcal{A}, \mathcal{B}$ then (X, Y) takes values in $\mathcal{A} \times \mathcal{B}$, and its entropy

$$H(X, Y) = - \sum_{x \in \mathcal{A}} \sum_{y \in \mathcal{B}} \mathbb{P}(X = x, Y = y) \log \mathbb{P}(X = x, Y = y)$$

is called the **joint entropy**. The same definition works for any finite tuple.

Lemma 2.22 (Subadditivity)

$H(X, Y) \leq H(X) + H(Y)$, with equality if and only if X and Y are independent.

Proof. Write $\mathcal{A} = \{x_1, \dots, x_m\}$, $\mathcal{B} = \{y_1, \dots, y_n\}$, and set $p_{ij} = \mathbb{P}(X = x_i, Y = y_j)$, $p_i = \mathbb{P}(X = x_i)$, $q_j = \mathbb{P}(Y = y_j)$. Both (p_{ij}) and $(p_i q_j)$ are probability distributions on $\mathcal{A} \times \mathcal{B}$, so Gibbs' inequality applies and gives

$$\begin{aligned} H(X, Y) &= - \sum_{i,j} p_{ij} \log p_{ij} \leq - \sum_{i,j} p_{ij} \log(p_i q_j) \\ &= - \sum_i \left(\sum_j p_{ij} \right) \log p_i - \sum_j \left(\sum_i p_{ij} \right) \log q_j \\ &= - \sum_i p_i \log p_i - \sum_j q_j \log q_j = H(X) + H(Y). \end{aligned}$$

Equality in Gibbs holds precisely when $p_{ij} = p_i q_j$ for all i, j , which is exactly independence. $\square$

Intuitively: knowing two things at once is never harder than knowing each separately, and it is exactly as hard only when the two are unrelated.

§3 Noisy Channels and Error-Correcting Codes

We now switch the channel back on. Throughout this section, unless we say otherwise, the channel is the binary symmetric channel with error probability $p < \frac{1}{2}$, and a code is a subset of $\mathbb{F}_2^n$.

Definition 3.1 (Binary Code)

A **binary** $[n, m]$ -**code** is a subset $C \subseteq \{0, 1\}^n$ with $|C| = m$. We call n the **length** of the code and the elements of C its **codewords**.

Such a code lets us send one of m messages using n bits, that is, using the channel n times. Since $1 \leq m \leq 2^n$, the information rate $\rho(C) = \frac{1}{n} \log m$ satisfies $0 \leq \rho(C) \leq 1$, with $\rho(C) = 0$ for the useless code $|C| = 1$ and $\rho(C) = 1$ for the code $C = \{0, 1\}^n$ which does no error correction at all. All the interest lies in between.

§3.1 Decoding Rules

Suppose Bob receives $x \in \{0, 1\}^n$. What should he guess was sent? There are three natural answers.

Definition 3.2 (Decoding Rules)

Let C be a binary $[n, m]$ -code.

- The **ideal observer** rule decodes x as the codeword $c \in C$ maximising $\mathbb{P}(c \text{ sent} \mid x \text{ received})$.
- The **maximum likelihood** rule decodes x as the $c \in C$ maximising $\mathbb{P}(x \text{ received} \mid c \text{ sent})$.
- The **minimum distance** rule decodes x as the $c \in C$ minimising the Hamming distance $d(x, c)$,

$$d(x, y) = |\{i : x_i \neq y_i\}|.$$

The first is what we actually want, but it requires knowing the prior distribution on messages. The third is what we can actually compute. Happily, under mild hypotheses they all coincide.

Lemma 3.3

Let C be a binary $[n, m]$ -code used over a BSC with error probability p .

- If all messages are equally likely, the ideal observer and maximum likelihood rules agree.
- If $p < \frac{1}{2}$, the maximum likelihood and minimum distance rules agree.

Proof. (i). By Bayes' theorem,

$$\mathbb{P}(c \text{ sent} \mid x \text{ received}) = \frac{\mathbb{P}(c \text{ sent})}{\mathbb{P}(x \text{ received})} \mathbb{P}(x \text{ received} \mid c \text{ sent}).$$

For fixed x , the denominator does not depend on c , and by hypothesis neither does $\mathbb{P}(c \text{ sent})$. So the two quantities are maximised together.

(ii). If $d(x, c) = r$ then exactly r of the n transmitted bits were flipped, so

$$\mathbb{P}(x \text{ received} \mid c \text{ sent}) = p^r (1 - p)^{n-r} = (1 - p)^n \left(\frac{p}{1 - p} \right)^r.$$

As $p < \frac{1}{2}$ we have $\frac{p}{1-p} < 1$, so this is a strictly decreasing function of r . Maximising it is therefore the same as minimising $r = d(x, c)$. $\square$

The hypothesis in (i) is reasonable in practice, because if we first compress the message using a good noiseless code the resulting bits look close to uniform. The hypothesis in (ii) is harmless, as discussed earlier. Channels with $p = 0$ are called **lossless**, and channels with $p = \frac{1}{2}$ are called **useless**.

It is worth seeing that the rules really can differ.

Example 3.4

Let $C = \{000, 111\}$, sent with probabilities $\alpha = \frac{9}{10}$ and $1 - \alpha = \frac{1}{10}$ respectively, over a BSC with $p = \frac{1}{4}$. Suppose 110 is received. Then

$$\mathbb{P}(000 \text{ sent} \mid 110) = \frac{\alpha p^2(1-p)}{\alpha p^2(1-p) + (1-\alpha)p(1-p)^2} = \frac{\frac{9}{10} \cdot \frac{1}{16} \cdot \frac{3}{4}}{\frac{9}{10} \cdot \frac{1}{16} \cdot \frac{3}{4} + \frac{1}{10} \cdot \frac{1}{4} \cdot \frac{9}{16}} = \frac{3}{4},$$

so $\mathbb{P}(111 \text{ sent} \mid 110) = \frac{1}{4}$. The ideal observer decodes 110 as 000, since 000 was so much more likely *a priori*. But $d(110, 111) = 1 < 2 = d(110, 000)$, so minimum distance decoding – and hence maximum likelihood decoding – gives 111.

Remark. Minimum distance decoding is conceptually simple but potentially expensive: naively one compares the received word against every codeword, which is hopeless once $|C|$ is large. Much of Section ?? is devoted to making it cheap for structured codes. One should also fix a convention for ties: either make an arbitrary choice, or declare a failure and ask for retransmission.

From now on we use minimum distance decoding.

§3.2 Error Detection and Correction

Definition 3.5

A binary $[n, m]$ -code C is

- **d -error detecting** if changing at most d digits of a codeword can never produce another codeword;
- **e -error correcting** if, knowing that $x \in \mathbb{F}_2^n$ differs from some codeword in at most e places, we can determine that codeword.

Example 3.6 (Repetition Code)

The **repetition code** of length n has codewords $00 \dots 0$ and $11 \dots 1$. It is an $[n, 2]$ -code, it is $(n - 1)$ -error detecting and $\lfloor \frac{n-1}{2} \rfloor$ -error correcting, and its information rate is $\frac{1}{n}$, which tends to 0. Reliability is bought at the price of everything else.

Example 3.7 (Simple Parity Check Code)

The **simple parity check code** (or **paper tape code**) of length n is

$$C = \left\{ (x_1, \dots, x_n) \in \mathbb{F}_2^n : \sum_{i=1}^n x_i = 0 \right\}.$$

This is an $[n, 2^{n-1}]$ -code, it is 1-error detecting and 0-error correcting, and its information rate is $\frac{n-1}{n} \rightarrow 1$. Economy is bought at the price of everything else.

Between these two extremes sits the first genuinely clever code.

Example 3.8 (Hamming's Original Code)

Define $C \subseteq \mathbb{F}_2^7$ by the three conditions

$$c_1 + c_3 + c_5 + c_7 = 0, \quad c_2 + c_3 + c_6 + c_7 = 0, \quad c_4 + c_5 + c_6 + c_7 = 0.$$

The digits c_3, c_5, c_6, c_7 may be chosen freely and then c_1, c_2, c_4 are determined, so $|C| = 2^4 = 16$ and C is a $[7, 16]$ -code with rate $\frac{4}{7}$.

Suppose $x \in \mathbb{F}_2^7$ is received. Form the **syndrome** $z = (z_1, z_2, z_4)$ where

$$z_1 = x_1 + x_3 + x_5 + x_7, \quad z_2 = x_2 + x_3 + x_6 + x_7, \quad z_4 = x_4 + x_5 + x_6 + x_7.$$

If $x \in C$ then $z = (0, 0, 0)$. If instead $x = c + e_i$, a codeword with the i th digit flipped, then by linearity $z_x = z_{e_i}$, and one checks case by case that

$$z_1 + 2z_2 + 4z_4 = i$$

as an ordinary integer. For instance e_5 appears in the first and third conditions but not the second, giving $z = (1, 0, 1)$ and $1 + 0 + 4 = 5$. So the syndrome does not merely tell us that an error has occurred, it tells us *where*: the code is 1-error correcting. This is a remarkably efficient piece of bookkeeping, and we will see in a moment that in a precise sense it cannot be improved on.

To analyse such codes properly we need the geometry of $\mathbb{F}_2^n$.

Lemma 3.9

The Hamming distance d is a metric on $\mathbb{F}_2^n$.

Proof. Certainly $d(x, y) \geq 0$ with equality if and only if $x = y$, and $d(x, y) = d(y, x)$. For the triangle inequality, note that if $x_i \neq z_i$ then $x_i \neq y_i$ or $y_i \neq z_i$, so

$$\{i : x_i \neq z_i\} \subseteq \{i : x_i \neq y_i\} \cup \{i : y_i \neq z_i\},$$

and taking sizes gives $d(x, z) \leq d(x, y) + d(y, z)$. □

Equivalently, $d(x, y) = \sum_i d_1(x_i, y_i)$ where d_1 is the discrete metric on $\mathbb{F}_2$; the triangle inequality is inherited coordinatewise.

Definition 3.10 (Minimum Distance)

The **minimum distance** of a code C with $|C| \geq 2$ is

$$d(C) = \min\{d(c_1, c_2) : c_1, c_2 \in C, c_1 \neq c_2\}.$$

An $[n, m]$ -code with minimum distance d is called an $[n, m, d]$ -code.

Lemma 3.11

Let C be a code with minimum distance d . Then

- (i) C is $(d - 1)$ -error detecting, but cannot detect all sets of d errors;
- (ii) C is $\lfloor \frac{d-1}{2} \rfloor$ -error correcting, but cannot correct all sets of $\lfloor \frac{d-1}{2} \rfloor + 1$ errors.

Proof. (i). If $c \in C$ and $1 \leq d(x, c) \leq d - 1$ then $x \notin C$, since every other codeword is at distance at least d from c ; so the corruption is detected. On the other hand if c_1, c_2 are codewords at distance exactly d , then d well-chosen errors carry c_1 to c_2 and nothing at all is noticed.

(ii). Put $e = \lfloor \frac{d-1}{2} \rfloor$, so that $2e < d$. Suppose $x \in \mathbb{F}_2^n$ and $c_1 \in C$ with $d(x, c_1) \leq e$. For any other codeword c_2 , the triangle inequality gives

$$d(x, c_2) \geq d(c_1, c_2) - d(x, c_1) \geq d - e > e,$$

so c_1 is the unique codeword within distance e of x and we recover it. For the second half, choose c_1, c_2 with $d(c_1, c_2) = d$ and let x agree with c_1 except in $e + 1$ of the places where c_1 and c_2 differ. Then $d(x, c_1) = e + 1$ while $d(x, c_2) = d - (e + 1) \leq e + 1$, using $d \leq 2(e + 1)$; so x is at least as close to c_2 as to c_1 and we cannot reliably decode. $\square$

Example 3.12

The repetition code of length n is an $[n, 2, n]$ -code. The simple parity check code of length n is an $[n, 2^{n-1}, 2]$ -code, since flipping any two digits preserves the parity. The trivial code $\mathbb{F}_2^n$ is an $[n, 2^n, 1]$ -code. Hamming's original code is 1-error correcting so has $d \geq 3$ by Lemma 3.11; and 0000000 and 1110000 are both codewords, so $d = 3$ exactly, making it a $[7, 16, 3]$ -code.

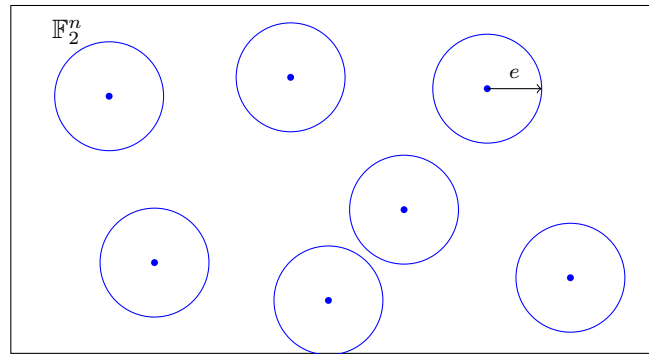
§3.3 Covering Estimates and Perfect Codes

The picture behind Lemma 3.11 is that an e -error correcting code is a packing of disjoint balls of radius e into $\mathbb{F}_2^n$. Counting gives an immediate upper bound on how many codewords there can be.

Definition 3.13

For $x \in \mathbb{F}_2^n$ and $r \geq 0$, write $B(x, r) = \{y \in \mathbb{F}_2^n : d(x, y) \leq r\}$ for the closed **Hamming ball** of radius r . Its size does not depend on x , and we write

$$V(n, r) = |B(x, r)| = \sum_{i=0}^r \binom{n}{i}.$$



The dots are the codewords; the discs are the balls of radius e inside which minimum distance decoding succeeds. They must be disjoint, and they all live inside a space of size 2^n .

Lemma 3.14 (Hamming's Bound; the Sphere-Packing Bound)

An e -error correcting code C of length n satisfies

$$|C| \leq \frac{2^n}{V(n, e)}.$$

Proof. If $c_1 \neq c_2$ are codewords then $B(c_1, e) \cap B(c_2, e) = \emptyset$, since a point in the intersection could not be decoded. Hence

$$\sum_{c \in C} |B(c, e)| \leq |\mathbb{F}_2^n|, \quad \text{that is} \quad |C| V(n, e) \leq 2^n. \quad \square$$

Definition 3.15 (Perfect Code)

An e -error correcting code C of length n with $|C| = \frac{2^n}{V(n, e)}$ is called **perfect**.

Remark. Equivalently, C is perfect if the balls $B(c, e)$ for $c \in C$ partition $\mathbb{F}_2^n$: every word in the space is within distance e of exactly one codeword. Perfect codes are wonderfully efficient but also brittle, since any $e+1$ errors are guaranteed to be decoded to the wrong codeword, with no warning.

Example 3.16

Hamming's $[7, 16, 3]$ -code is 1-error correcting, and

$$\frac{2^7}{V(7, 1)} = \frac{128}{1+7} = 16 = |C|,$$

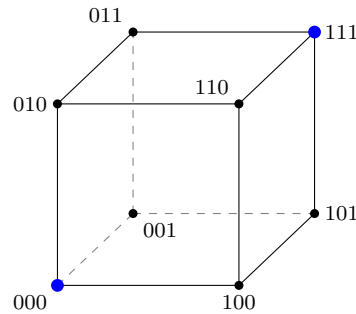
so it is perfect. Indeed, the syndrome map sends $\mathbb{F}_2^7$ onto $\mathbb{F}_2^3$, and the eight syndromes correspond precisely to 'no error' and the seven possible single-bit errors.

Example 3.17

The repetition code of length n is perfect if and only if n is odd. If $n = 2k + 1$ it is k -error correcting and $V(n, k) = \sum_{i \leq k} \binom{n}{i} = 2^{n-1}$ by symmetry, so $2^n / V(n, k) =$

$2 = |C|$. If n is even the count fails.

The smallest interesting case is worth drawing. Here is $\mathbb{F}_2^3$ as the vertices of a cube, with the repetition code $\{000, 111\}$ marked.



Every vertex is adjacent to exactly one of the two blue vertices, which is the statement that the $[3, 2, 3]$ repetition code is perfect.

Remark. If $\frac{2^n}{V(n,e)}$ is not an integer then certainly no perfect e -error correcting code of length n exists. The converse is false: with $n = 90$ and $e = 2$ the quotient is an integer, but no such code exists.

Definition 3.18

$A(n, d)$ denotes the largest m for which an $[n, m, d]$ -code exists.

Determining $A(n, d)$ in general is a hard open problem, and only small cases are known.

Example 3.19

$A(n, 1) = 2^n$, achieved by the trivial code. $A(n, 2) = 2^{n-1}$, achieved by the simple parity check code (the upper bound follows from the next theorem). $A(n, n) = 2$, achieved by the repetition code.

Lemma 3.20

$A(n, d+1) \leq A(n, d)$.

Proof. Let $m = A(n, d+1)$ and let C be an $[n, m, d+1]$ -code. Choose distinct $c_1, c_2 \in C$ with $d(c_1, c_2) = d+1$, and let c'_1 differ from c_1 in exactly one of the places where c_1 and c_2 differ, so $d(c'_1, c_2) = d$. For any other $c \in C$,

$$d+1 \leq d(c, c_1) \leq d(c, c'_1) + d(c'_1, c_1) = d(c, c'_1) + 1,$$

so $d(c, c'_1) \geq d$. Hence $C' = (C \setminus \{c_1\}) \cup \{c'_1\}$ has m elements, length n , and minimum distance exactly d . $\square$

Corollary 3.21

$A(n, d) = \max\{m : \text{there is an } [n, m, d']\text{-code for some } d' \geq d\}$.

Now we can sandwich $A(n, d)$ between two natural bounds.

Theorem 3.22 (Hamming and GSV Bounds)

$$\frac{2^n}{V(n, d-1)} \leq A(n, d) \leq \frac{2^n}{V(n, \lfloor \frac{d-1}{2} \rfloor)}.$$

The upper bound is Hamming's bound, a *packing* estimate. The lower bound is the **Gilbert–Shannon–Varshamov** (GSV) bound, and is a *covering* estimate: it says that a maximal code cannot leave any large empty region.

Proof. The upper bound is Lemma 3.14 together with Lemma 3.11(ii).

For the lower bound, let $m = A(n, d)$ and let C be an $[n, m, d]$ -code. There can be no $x \in \mathbb{F}_2^n$ with $d(x, c) \geq d$ for all $c \in C$, for then $C \cup \{x\}$ would be an $[n, m+1, d]$ -code, contradicting maximality. Hence every x lies within distance $d-1$ of some codeword, that is,

$$\mathbb{F}_2^n = \bigcup_{c \in C} B(c, d-1),$$

and counting gives $2^n \leq m V(n, d-1)$. $\square$

There is a third bound, of a completely different flavour, which is often the sharpest of all for large d .

Theorem 3.23 (Singleton Bound)

For any $[n, m, d]$ -code, $m \leq 2^{n-d+1}$.

Proof. Let C be an $[n, m, d]$ -code and consider the map $\pi: C \rightarrow \mathbb{F}_2^{n-d+1}$ which deletes the last $d-1$ coordinates. If $c_1 \neq c_2$ had the same image then they would agree in the first $n-d+1$ places and so differ in at most $d-1$ places, contradicting $d(C) = d$. So π is injective and $m = |C| \leq 2^{n-d+1}$. $\square$

A code attaining the Singleton bound is called **maximum distance separable**, or MDS. Over $\mathbb{F}_2$ these are rare, but we shall meet a large family of them over larger alphabets in Section ??.

Example 3.24

Take $n = 10$, $d = 3$. Then $V(10, 1) = 11$ and $V(10, 2) = 1 + 10 + 45 = 56$, so Theorem 3.22 gives

$$\frac{1024}{56} \leq A(10, 3) \leq \frac{1024}{11}, \quad \text{that is} \quad 19 \leq A(10, 3) \leq 93.$$

Singleton gives only $A(10, 3) \leq 2^8 = 256$, which is worse here. The true value, established by computer search, is $A(10, 3) = 72$; so the GSV bound in particular is far from sharp.

§3.4 Asymptotics

For applications we care about long codes, and about how the rate behaves when we insist that the minimum distance grows proportionally to the length. So fix $\delta \in (0, \frac{1}{2})$

and study

$$\frac{1}{n} \log A(n, \lfloor n\delta \rfloor)$$

as $n \rightarrow \infty$. The answer is governed, once again, by the binary entropy function.

Proposition 3.25

Let $0 < \delta < \frac{1}{2}$. Then

- (i) $\log V(n, \lfloor n\delta \rfloor) \leq nH(\delta)$;
- (ii) $\frac{1}{n} \log A(n, \lfloor n\delta \rfloor) \geq 1 - H(\delta)$.

Proof. (i) implies (ii). By the GSV bound and monotonicity of V in its second argument,

$$A(n, \lfloor n\delta \rfloor) \geq \frac{2^n}{V(n, \lfloor n\delta \rfloor - 1)} \geq \frac{2^n}{V(n, \lfloor n\delta \rfloor)},$$

so taking logarithms and using (i),

$$\frac{1}{n} \log A(n, \lfloor n\delta \rfloor) \geq 1 - \frac{\log V(n, \lfloor n\delta \rfloor)}{n} \geq 1 - H(\delta).$$

Proof of (i). Since H is increasing on $(0, \frac{1}{2})$ and $V(n, \lfloor n\delta \rfloor) = V(n, \lfloor n\delta' \rfloor)$ for $\delta' = \lfloor n\delta \rfloor/n \leq \delta$, we may assume $n\delta$ is an integer. Now expand $1 = (\delta + (1 - \delta))^n$ and throw away all but the first $n\delta + 1$ terms:

$$\begin{aligned} 1 &= \sum_{i=0}^n \binom{n}{i} \delta^i (1 - \delta)^{n-i} \geq \sum_{i=0}^{n\delta} \binom{n}{i} \delta^i (1 - \delta)^{n-i} \\ &= (1 - \delta)^n \sum_{i=0}^{n\delta} \binom{n}{i} \left(\frac{\delta}{1 - \delta} \right)^i \geq (1 - \delta)^n \left(\frac{\delta}{1 - \delta} \right)^{n\delta} \sum_{i=0}^{n\delta} \binom{n}{i} \\ &= \delta^{n\delta} (1 - \delta)^{n(1-\delta)} V(n, n\delta), \end{aligned}$$

where the second inequality uses $\frac{\delta}{1-\delta} < 1$ (as $\delta < \frac{1}{2}$), so that each of the terms $\left(\frac{\delta}{1-\delta}\right)^i$ with $i \leq n\delta$ is at least $\left(\frac{\delta}{1-\delta}\right)^{n\delta}$. Taking logarithms,

$$0 \geq n\delta \log \delta + n(1 - \delta) \log(1 - \delta) + \log V(n, n\delta) = -nH(\delta) + \log V(n, n\delta),$$

which is (i). □

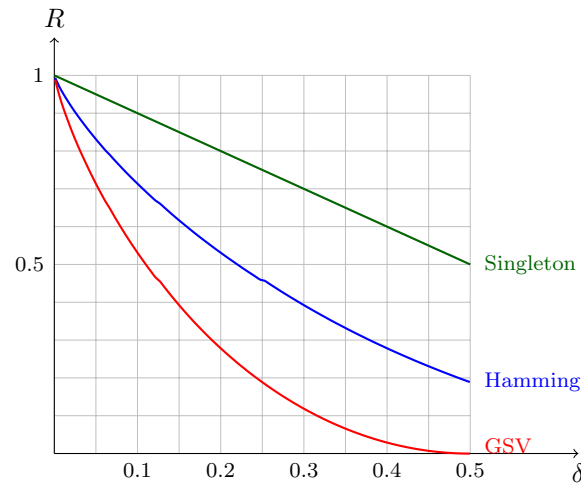
Lemma 3.26

For $0 < \delta < \frac{1}{2}$, $\lim_{n \rightarrow \infty} \frac{\log V(n, \lfloor n\delta \rfloor)}{n} = H(\delta)$.

Proof. The upper bound is Proposition 3.25(i). For the lower bound it suffices to keep the single term $\binom{n}{\lfloor n\delta \rfloor}$ and apply Stirling's formula, which gives $\frac{1}{n} \log \binom{n}{\lfloor n\delta \rfloor} \rightarrow H(\delta)$. □

So the constant $H(\delta)$ in the proposition cannot be improved. It is instructive to plot the

three bounds we now have, expressed as constraints on the rate $R = \frac{1}{n} \log|C|$ in terms of the relative distance $\delta = d/n$. The Hamming bound becomes $R \leq 1 - H(\delta/2)$, the Singleton bound becomes $R \leq 1 - \delta$, and the GSV bound becomes $R \geq 1 - H(\delta)$.



Anything achievable lies below the blue and green curves and, by GSV, at least the region below the red curve *is* achievable. The gap between red and blue is enormous, and closing it – determining the true asymptotic trade-off between rate and distance – remains one of the central open problems of coding theory.

§3.5 Constructing New Codes from Old

We finish this section with three easy operations on codes; each is worth knowing because they let us move around the parameter space by hand. Let C be an $[n, m, d]$ -code.

Example 3.27 (Parity Check Extension)

The **parity check extension** of C is

$$C^+ = \left\{ \left(c_1, \dots, c_n, \sum_{i=1}^n c_i \right) : (c_1, \dots, c_n) \in C \right\},$$

an $[n + 1, m, d']$ -code where $d' = d + 1$ if d is odd and $d' = d$ if d is even. (All codewords of C^+ have even weight, so distances between them are even; hence an odd d must increase.)

Example 3.28 (Punctured Code)

Deleting the i th digit from every codeword gives the **punctured code** C^- . If $d \geq 2$ this is an $[n - 1, m, d']$ -code with $d' \in \{d - 1, d\}$; the codewords remain distinct because they differed in at least two places.

Example 3.29 (Shortened Code)

Fix i and $\alpha \in \mathbb{F}_2$, and let

$$C' = \{(c_1, \dots, c_{i-1}, c_{i+1}, \dots, c_n) : (c_1, \dots, c_{i-1}, \alpha, c_{i+1}, \dots, c_n) \in C\}.$$

This is the **shortened code**. It has length $n - 1$ and minimum distance $d' \geq d$, and for a suitable choice of α (namely, the more popular value of the i th digit) it has size $m' \geq \frac{m}{2}$.

§4 Information Theory

We now have two loose ends to tie up. In Section 2 we measured the cost of encoding a *single* symbol and found the entropy $H(X)$ standing in the way; but a real source emits a long stream of symbols, possibly with dependence between them, and we should ask what the cost per symbol is in the long run. In Section 3 we defined the capacity of a channel and did not compute it for a single example. Both questions are answered by the same circle of ideas, and the answers are Shannon's two coding theorems.

§4.1 Sources and Information Rate

Definition 4.1 (Source)

A **source** is a sequence of random variables $X_1, X_2, \dots$ taking values in an alphabet $\mathcal{A}$.

Example 4.2

A **Bernoulli** or **memoryless** source is one in which the X_i are independent and identically distributed. English text is emphatically not of this kind – the letter u is much more likely after a q – but it is the case in which everything can be computed.

Definition 4.3 (Reliable Encoding)

A source $X_1, X_2, \dots$ is **reliably encodable at rate r** if there are subsets $A_n \subseteq \mathcal{A}^n$ with

- (i) $\lim_{n \rightarrow \infty} \frac{\log |A_n|}{n} = r$, and
- (ii) $\lim_{n \rightarrow \infty} \mathbb{P}((X_1, \dots, X_n) \in A_n) = 1$.

The **information rate** H of the source is the infimum of all rates at which it is reliably encodable.

The idea is that we agree to encode faithfully only the messages in A_n , which takes $\log |A_n|$ bits, and accept a vanishing probability of complete failure otherwise. Since $A_n = \mathcal{A}^n$ always works, $H \leq \log |\mathcal{A}|$; and clearly $H \geq 0$. Both extremes occur.

Before going further, a piece of notation which is easy to misread. If X is a discrete random variable with probability mass function $p_X(x) = \mathbb{P}(X = x)$, then $p(X)$ denotes the composite $p_X \circ X$, that is, the random variable

$$\omega \mapsto \mathbb{P}(X = X(\omega)).$$

It is the ‘probability of what actually happened’. Similarly for a tuple $X^{(n)} = (X_1, \dots, X_n)$ we write $p(X^{(n)})$ for the random variable $\omega \mapsto \mathbb{P}(X_1 = X_1(\omega), \dots, X_n = X_n(\omega))$. With this notation, $H(X) = \mathbb{E}[-\log p(X)]$.

Example 4.4

Let $\mathcal{A} = \{A, B, C\}$ and suppose $X^{(2)} = (X_1, X_2)$ takes the values AB, AC, BC, BA, CA, CB with probabilities 0.3, 0.1, 0.1, 0.2, 0.25, 0.05. Then $p_{X^{(2)}}(AB) = 0.3$ and so on, so the random variable $p(X^{(2)})$ takes the value 0.3 with probability 0.3, the value 0.1 with probability 0.2 (the two ways of getting a probability-0.1 outcome), the value 0.2 with probability 0.2, and so on.

Recall from IA Probability that $X_n \rightarrow L$ **in probability**, written $X_n \xrightarrow{\mathbb{P}} L$, if $\mathbb{P}(|X_n - L| > \varepsilon) \rightarrow 0$ for every $\varepsilon > 0$, and that the weak law of large numbers says that if $Y_1, Y_2, \dots$ are i.i.d. real random variables with finite mean then $\frac{1}{n} \sum_{i=1}^n Y_i \xrightarrow{\mathbb{P}} \mathbb{E}[Y_1]$.

Example 4.5

Let $X_1, X_2, \dots$ be a Bernoulli source. Then $p(X_1), p(X_2), \dots$ are i.i.d., and by independence

$$p(X_1, \dots, X_n) = p(X_1)p(X_2) \cdots p(X_n).$$

Applying the weak law to the i.i.d. variables $-\log p(X_i)$,

$$-\frac{1}{n} \log p(X_1, \dots, X_n) = -\frac{1}{n} \sum_{i=1}^n \log p(X_i) \xrightarrow{\mathbb{P}} \mathbb{E}[-\log p(X_1)] = H(X_1).$$

This last observation is the key to everything in this section, and it deserves a name; but first let us see that it gives the right answer for Bernoulli sources by an elementary route.

Lemma 4.6

The information rate of a Bernoulli source $X_1, X_2, \dots$ is at most the expected word length of an optimal binary code c for X_1 .

Proof. Let L_i be the length of the codeword $c(X_i)$, so the L_i are i.i.d. with mean $\mathbb{E}[L_1]$. Fix $\varepsilon > 0$ and let

$$A_n = \{x \in \mathcal{A}^n : c^*(x) \text{ has length} < n(\mathbb{E}[L_1] + \varepsilon)\}.$$

Then

$$\mathbb{P}((X_1, \dots, X_n) \in A_n) = \mathbb{P}\left(\frac{1}{n} \sum_{i=1}^n L_i < \mathbb{E}[L_1] + \varepsilon\right) \rightarrow 1$$

by the weak law. Since c is decipherable, c^* is injective, and there are fewer than $2^{n(\mathbb{E}[L_1] + \varepsilon) + 1}$ binary strings of length less than $n(\mathbb{E}[L_1] + \varepsilon)$; so

$$|A_n| \leq 2^{n(\mathbb{E}[L_1] + \varepsilon) + 1}.$$

Enlarging A_n if necessary so that $|A_n| = \lfloor 2^{n(\mathbb{E}[L_1] + \varepsilon)} \rfloor$ (which only helps condition

(ii)), we get $\frac{1}{n} \log |A_n| \rightarrow \mathbb{E}[L_1] + \varepsilon$. Hence the source is reliably encodable at rate $\mathbb{E}[L_1] + \varepsilon$ for every $\varepsilon > 0$, and the result follows. $\square$

Corollary 4.7

A Bernoulli source has information rate $H < H(X_1) + 1$.

Proof. Combine Lemma 4.6 with the upper bound in Shannon’s noiseless coding theorem (Theorem 2.15). $\square$

The ‘+1’ is the same waste we met before, and the same trick removes it. Suppose we encode in blocks of size N : set $Y_1 = (X_1, \dots, X_N)$, $Y_2 = (X_{N+1}, \dots, X_{2N})$ and so on, giving a source over the alphabet $\mathcal{A}^N$. One checks directly from the definition that if (X_i) has information rate H then (Y_j) has information rate NH .

Proposition 4.8

The information rate H of a Bernoulli source satisfies $H \leq H(X_1)$.

Proof. The blocked source (Y_j) is again Bernoulli, so Corollary 4.7 applies to it:

$$NH < H(Y_1) + 1 = H(X_1, \dots, X_N) + 1 = NH(X_1) + 1,$$

the last equality by independence and Lemma 2.22 (with equality since the X_i are independent). Dividing by N gives $H < H(X_1) + \frac{1}{N}$ for every N . $\square$

§4.2 The Asymptotic Equipartition Property

Example 4.5 isolates exactly the property we need, and it is worth abstracting.

Definition 4.9 (AEP)

A source $X_1, X_2, \dots$ satisfies the **asymptotic equipartition property** (AEP) with constant $H \geq 0$ if

$$-\frac{1}{n} \log p(X_1, \dots, X_n) \xrightarrow{\mathbb{P}} H.$$

Example 4.10

Toss a biased coin with $\mathbb{P}(\text{head}) = q$, and let $X_1, X_2, \dots$ record the results. After N tosses we expect about qN heads, and any *particular* sequence with exactly qN heads has probability

$$q^{qN} (1-q)^{(1-q)N} = 2^{N(q \log q + (1-q) \log(1-q))} = 2^{-NH(q)}.$$

Not every sequence looks like this; but the atypical ones carry vanishing total probability. So with high probability we see a ‘typical sequence’, and typical sequences are all roughly *equally likely*, each with probability about $2^{-NH(q)}$. This is what the name is describing: the probability mass is asymptotically spread evenly over about 2^{NH} sequences.

That reformulation is worth recording precisely.

Lemma 4.11

A source satisfies the AEP with constant H if and only if for every $\varepsilon > 0$ there is n_0 such that for all $n \geq n_0$ there is a **typical set** $T_n \subseteq \mathcal{A}^n$ with

- (i) $\mathbb{P}((X_1, \dots, X_n) \in T_n) > 1 - \varepsilon$;
- (ii) $2^{-n(H+\varepsilon)} \leq p(x_1, \dots, x_n) \leq 2^{-n(H-\varepsilon)}$ for all $(x_1, \dots, x_n) \in T_n$.

Proof. Suppose the AEP holds and let $\varepsilon > 0$. Define

$$T_n = \left\{ (x_1, \dots, x_n) : \left| -\frac{1}{n} \log p(x_1, \dots, x_n) - H \right| \leq \varepsilon \right\},$$

which is precisely the set on which (ii) holds. By convergence in probability, $\mathbb{P}((X_1, \dots, X_n) \in T_n) \rightarrow 1$, so (i) holds for n large.

Conversely, given such sets T_n , for any $\varepsilon > 0$ we have

$$\mathbb{P}\left(\left| -\frac{1}{n} \log p(X_1, \dots, X_n) - H \right| \leq \varepsilon\right) \geq \mathbb{P}((X_1, \dots, X_n) \in T_n) > 1 - \varepsilon$$

for $n \geq n_0(\varepsilon)$, which is convergence in probability. $\square$

§4.3 Shannon's First Coding Theorem

Theorem 4.12 (Shannon's First Coding Theorem)

A source satisfying the AEP with constant H has information rate exactly H .

Proof. Rate H is achievable. Let $\varepsilon > 0$ and take typical sets T_n as in Lemma 4.11. For $n \geq n_0(\varepsilon)$ and $(x_1, \dots, x_n) \in T_n$ we have $p(x_1, \dots, x_n) \geq 2^{-n(H+\varepsilon)}$, so

$$1 \geq \mathbb{P}(T_n) \geq |T_n| 2^{-n(H+\varepsilon)}, \quad \text{giving} \quad \frac{1}{n} \log |T_n| \leq H + \varepsilon.$$

Taking $A_n = T_n$ (padded if necessary so that the limit in (i) exists and equals $H + \varepsilon$) shows that the source is reliably encodable at rate $H + \varepsilon$. As $\varepsilon > 0$ was arbitrary, the information rate is at most H .

No smaller rate is achievable. If $H = 0$ there is nothing to prove, so assume $H > 0$ and let $0 < \varepsilon < \frac{H}{2}$. Suppose for contradiction that the source is reliably encodable at rate $H - 2\varepsilon$, witnessed by sets A_n . Let T_n be typical sets for this ε . For $(x_1, \dots, x_n) \in T_n$ we have $p(x_1, \dots, x_n) \leq 2^{-n(H-\varepsilon)}$, so

$$\mathbb{P}((X_1, \dots, X_n) \in A_n \cap T_n) \leq |A_n| 2^{-n(H-\varepsilon)},$$

and hence

$$\frac{1}{n} \log \mathbb{P}(A_n \cap T_n) \leq -(H - \varepsilon) + \frac{1}{n} \log |A_n| \rightarrow -(H - \varepsilon) + (H - 2\varepsilon) = -\varepsilon.$$

Therefore $\log \mathbb{P}(A_n \cap T_n) \rightarrow -\infty$, so $\mathbb{P}(A_n \cap T_n) \rightarrow 0$. But

$$\mathbb{P}(T_n) \leq \mathbb{P}(A_n \cap T_n) + \mathbb{P}(\mathcal{A}^n \setminus A_n) \rightarrow 0 + 0 = 0,$$

contradicting $\mathbb{P}(T_n) > 1 - \varepsilon$. So the information rate is at least H . $\square$

Corollary 4.13

A Bernoulli source has information rate exactly $H(X_1)$.

Proof. Example 4.5 verifies the AEP with constant $H(X_1)$. $\square$

Remark. The theorem is not merely an existence statement; it comes with a coding scheme. Encode the typical sequences with a block code of length $\lceil n(H + \varepsilon) \rceil$, and encode the atypical sequences however you like (say, by a reserved escape symbol followed by the raw string). The scheme fails with vanishing probability and costs $H + \varepsilon$ bits per symbol.

Many non-Bernoulli sources also satisfy the AEP. Under suitable hypotheses – for instance for a stationary ergodic source – the sequence $\frac{1}{n}H(X_1, \dots, X_n)$ is decreasing, and the AEP holds with $H = \lim_n \frac{1}{n}H(X_1, \dots, X_n)$. This is what makes the theory applicable to real text.

§4.4 Capacity of the Binary Symmetric Channel

We now turn to the channel side. Recall the definitions from Section 1: a code of length n is $C \subseteq \mathcal{A}^n$, its rate is $\rho(C) = \frac{\log|C|}{n}$, its error rate is $\hat{e}(C) = \max_{c \in C} \mathbb{P}(\text{error} \mid c \text{ sent})$, and the capacity is the supremum of rates at which reliable transmission is possible.

As a warm-up – and because it shows exactly where the GSV bound earns its keep – let us prove directly that a reasonably good binary symmetric channel has positive capacity.

Lemma 4.14

Let $\varepsilon > 0$ and let a BSC with error probability p be used n times. Then

$$\lim_{n \rightarrow \infty} \mathbb{P}(\text{the channel makes at least } n(p + \varepsilon) \text{ errors}) = 0.$$

Proof. Let U_i be the indicator that the i th digit is mistransmitted. The U_i are i.i.d. with $\mathbb{E}[U_i] = p$, and

$$\mathbb{P}\left(\sum_{i=1}^n U_i \geq n(p + \varepsilon)\right) \leq \mathbb{P}\left(\left|\frac{1}{n} \sum_{i=1}^n U_i - p\right| \geq \varepsilon\right) \rightarrow 0$$

by the weak law of large numbers. $\square$

Proposition 4.15

A binary symmetric channel with error probability $p < \frac{1}{4}$ has nonzero capacity.

Proof. Choose δ with $2p < \delta < \frac{1}{2}$; this is possible precisely because $p < \frac{1}{4}$. We claim we can transmit reliably at rate $R = 1 - H(\delta) > 0$.

Let C_n be an $[n, A(n, \lfloor n\delta \rfloor), \lfloor n\delta \rfloor]$ -code of maximal size. By the GSV bound and Proposition 3.25,

$$|C_n| = A(n, \lfloor n\delta \rfloor) \geq 2^{n(1-H(\delta))} = 2^{nR},$$

and passing to a subcode we may assume $|C_n| = \lfloor 2^{nR} \rfloor$, still with minimum distance at least $\lfloor n\delta \rfloor$. Using minimum distance decoding, an error occurs only if the channel makes more than $\lfloor \frac{\lfloor n\delta \rfloor - 1}{2} \rfloor$ errors, so

$$\hat{e}(C_n) \leq \mathbb{P}\left(\text{at least } \frac{n\delta - 1}{2} \text{ errors in } n \text{ uses}\right).$$

Now pick $\varepsilon > 0$ with $p + \varepsilon < \frac{\delta}{2}$, which is possible as $2p < \delta$. For large n ,

$$\frac{n\delta - 1}{2} = n\left(\frac{\delta}{2} - \frac{1}{2n}\right) > n(p + \varepsilon),$$

so $\hat{e}(C_n) \leq \mathbb{P}(\text{at least } n(p + \varepsilon) \text{ errors}) \rightarrow 0$ by Lemma 4.14. Since $\rho(C_n) \rightarrow R$, we are done. $\square$

This is already remarkable, but the constant is wrong: the answer is not $1 - H(\delta)$ for some $\delta > 2p$, and the restriction $p < \frac{1}{4}$ is an artefact. To get the truth we need a more delicate measure of how much a channel destroys.

§4.5 Conditional Entropy and Fano's Inequality

Definition 4.16 (Conditional Entropy)

Let X, Y take values in $\mathcal{A}, \mathcal{B}$. The **conditional entropy** of X given $Y = y$ is

$$H(X | Y = y) = - \sum_{x \in \mathcal{A}} \mathbb{P}(X = x | Y = y) \log \mathbb{P}(X = x | Y = y),$$

and the **conditional entropy** of X given Y is

$$H(X | Y) = \sum_{y \in \mathcal{B}} \mathbb{P}(Y = y) H(X | Y = y).$$

Clearly $H(X | Y) \geq 0$. It measures how much uncertainty about X remains once Y is known.

Lemma 4.17 (Chain Rule)

$$H(X, Y) = H(X | Y) + H(Y).$$

Proof. Expanding the definition and writing $\mathbb{P}(X = x | Y = y)\mathbb{P}(Y = y) = \mathbb{P}(X = x, Y = y)$,

$$H(X | Y) = - \sum_y \sum_x \mathbb{P}(X = x, Y = y) \log \frac{\mathbb{P}(X = x, Y = y)}{\mathbb{P}(Y = y)}$$

$$\begin{aligned}
&= - \sum_y \sum_x \mathbb{P}(X = x, Y = y) \log \mathbb{P}(X = x, Y = y) \\
&\quad + \sum_y \left(\sum_x \mathbb{P}(X = x, Y = y) \right) \log \mathbb{P}(Y = y) \\
&= H(X, Y) + \sum_y \mathbb{P}(Y = y) \log \mathbb{P}(Y = y) = H(X, Y) - H(Y). \square
\end{aligned}$$

Example 4.18

Let X be uniform on $\{1, \dots, 6\}$, modelling a die, and let Y be 0 if X is even and 1 if X is odd. Then Y is a function of X , so $H(X, Y) = H(X) = \log 6$, and $H(Y) = \log 2 = 1$. The chain rule gives $H(X | Y) = \log 6 - \log 2 = \log 3$, which is right: knowing the parity leaves three equally likely values. Applying the chain rule the other way round, $H(Y | X) = H(X, Y) - H(X) = 0$, as it must be.

Corollary 4.19

$H(X | Y) \leq H(X)$, with equality if and only if X and Y are independent.

Proof. Combine the chain rule with subadditivity (Lemma 2.22): $H(X | Y) = H(X, Y) - H(Y) \leq H(X)$, with the same equality condition. $\square$

Everything extends to tuples: replacing X by $X^{(r)} = (X_1, \dots, X_r)$ and Y by $Y^{(s)}$, we may speak of $H(X_1, \dots, X_r | Y_1, \dots, Y_s)$. One must read such expressions carefully: $H(X, Y | Z)$ is the entropy of the *pair* (X, Y) given Z , not the entropy of X together with the entropy of Y given Z .

Lemma 4.20

$H(X | Y) \leq H(X | Y, Z) + H(Z)$.

Proof. Expand $H(X, Y, Z)$ in two ways using the chain rule:

$$H(Z | X, Y) + \underbrace{H(X | Y)}_{H(X, Y)} + H(Y) = H(X, Y, Z) = H(X | Y, Z) + \underbrace{H(Z | Y) + H(Y)}_{H(Y, Z)}.$$

Since $H(Z | X, Y) \geq 0$ we get $H(X | Y) \leq H(X | Y, Z) + H(Z | Y)$, and $H(Z | Y) \leq H(Z)$ by Corollary 4.19. $\square$

Now for the key estimate. Think of X as what was sent and Y as what was decoded; then $p = \mathbb{P}(X \neq Y)$ is the probability of error, and Fano's inequality bounds how much uncertainty can remain.

Proposition 4.21 (Fano's Inequality)

Let X, Y take values in an alphabet $\mathcal{A}$ with $|\mathcal{A}| = m \geq 2$, and let $p = \mathbb{P}(X \neq Y)$. Then

$$H(X | Y) \leq H(p) + p \log(m - 1).$$

Proof. Let Z be the indicator of the event $X \neq Y$, so $H(Z) = H(p)$. By Lemma 4.20 it suffices to show $H(X | Y, Z) \leq p \log(m-1)$.

If $Z = 0$ then $X = Y$, so $H(X | Y = y, Z = 0) = 0$. If $Z = 1$ then X is one of the $m-1$ values other than y , so $H(X | Y = y, Z = 1) \leq \log(m-1)$. Hence

$$\begin{aligned} H(X | Y, Z) &= \sum_{y \in \mathcal{A}} \sum_{z \in \{0,1\}} \mathbb{P}(Y = y, Z = z) H(X | Y = y, Z = z) \\ &\leq \sum_{y \in \mathcal{A}} \mathbb{P}(Y = y, Z = 1) \log(m-1) = \mathbb{P}(Z = 1) \log(m-1) = p \log(m-1). \square \end{aligned}$$

The two terms have a clean reading: $H(p)$ is the information needed to say *whether* an error occurred, and $p \log(m-1)$ is the information needed to say *which* error it was, in the worst case.

§4.6 Mutual Information and Information Capacity

Definition 4.22 (Mutual Information)

The **mutual information** of X and Y is

$$I(X; Y) = H(X) - H(X | Y).$$

By the chain rule $I(X; Y) = H(X) + H(Y) - H(X, Y)$, from which it is visibly symmetric in X and Y , and non-negative by subadditivity, with $I(X; Y) = 0$ if and only if X and Y are independent. It measures how much knowing one variable tells us about the other – exactly the quantity a channel ought to be judged by.

Definition 4.23 (Information Capacity)

Let a discrete memoryless channel have input alphabet $\mathcal{A}$ and output alphabet $\mathcal{B}$. For a random variable X on $\mathcal{A}$, let Y be the resulting output. The **information capacity** of the channel is

$$C = \max_X I(X; Y),$$

the maximum being over all distributions of X on $\mathcal{A}$.

The maximum is attained: I is a continuous function of the input distribution $(p_1, \dots, p_m)$, which ranges over the compact simplex $\{p_i \geq 0, \sum p_i = 1\}$. Note that the information capacity depends only on the channel matrix, and is a finite computation; whereas the (operational) capacity of Section 1 is defined by a limiting process over all possible codes. The content of Shannon's second theorem is that they agree.

Theorem 4.24 (Shannon's Noisy Coding Theorem)

For a discrete memoryless channel, the operational capacity equals the information capacity.

We shall prove the inequality 'operational $\leq$ information' in general, and the reverse inequality for the binary symmetric channel. First, let us see what the theorem buys us.

Example 4.25 (i) *The binary symmetric channel with error probability p .* Let $\mathbb{P}(X = 0) = \alpha$. Given X , the output differs from the input with probability p regardless, so $H(Y | X = 0) = H(Y | X = 1) = H(p)$ and hence $H(Y | X) = H(p)$. Also $\mathbb{P}(Y = 0) = \alpha(1 - p) + (1 - \alpha)p$. Therefore

$$C = \max_{\alpha} [H(Y) - H(Y | X)] = \max_{\alpha} [H(\alpha(1 - p) + (1 - \alpha)p) - H(p)] = 1 - H(p),$$

the maximum being at $\alpha = \frac{1}{2}$, when the argument of the outer H is $\frac{1}{2}$. So

$$C = 1 + p \log p + (1 - p) \log(1 - p).$$

If $p = 0$ or $p = 1$ then $C = 1$; if $p = \frac{1}{2}$ then $C = 0$, confirming that such a channel is useless.

(ii) *The binary erasure channel with erasure probability p .* Here it is easier to compute $I(Y; X) = H(X) - H(X | Y)$. With $\mathbb{P}(X = 0) = \alpha$, receiving 0 or 1 determines X , so $H(X | Y = 0) = H(X | Y = 1) = 0$, while $H(X | Y = \star) = H(\alpha)$. As $\mathbb{P}(Y = \star) = p$,

$$C = \max_{\alpha} [H(\alpha) - pH(\alpha)] = (1 - p) \max_{\alpha} H(\alpha) = 1 - p,$$

again at $\alpha = \frac{1}{2}$. This is exactly what one would guess: a fraction p of the transmissions is simply lost.

Note the strength of the first computation. A channel that flips one bit in ten has capacity $1 - H(0.1) = 0.531$, so we can push more than half a bit of genuine information through each use of it, with arbitrarily small error probability.

Lemma 4.26

If a discrete memoryless channel has information capacity C , its n th extension has information capacity nC .

Proof. Let $X_1, \dots, X_n$ be the input and $Y_1, \dots, Y_n$ the output. Because the channel is memoryless, Y_i depends only on X_i , so

$$H(Y_1, \dots, Y_n | X_1, \dots, X_n) = \sum_{i=1}^n H(Y_i | X_1, \dots, X_n) = \sum_{i=1}^n H(Y_i | X_i).$$

Hence, using subadditivity for the first term,

$$\begin{aligned} I(X_1, \dots, X_n; Y_1, \dots, Y_n) &= H(Y_1, \dots, Y_n) - H(Y_1, \dots, Y_n | X_1, \dots, X_n) \\ &\leq \sum_{i=1}^n H(Y_i) - \sum_{i=1}^n H(Y_i | X_i) = \sum_{i=1}^n I(X_i; Y_i) \leq nC. \end{aligned}$$

Conversely, take the X_i independent and each achieving $I(X_i; Y_i) = C$. Then the Y_i are independent too, so $H(Y_1, \dots, Y_n) = \sum_i H(Y_i)$ and the first inequality is an equality. Hence the maximum is exactly nC . $\square$

§4.7 Proof of the Noisy Coding Theorem

We begin with the converse, which holds for any discrete memoryless channel.

Proposition 4.27 (Converse to the Noisy Coding Theorem)

The operational capacity of a discrete memoryless channel is at most its information capacity C .

Proof. Suppose, for contradiction, that reliable transmission is possible at some rate $R > C$. Then there are codes C_n of length n and size $|C_n| = 2^{nR}$ with $\rho(C_n) \rightarrow R$ and $\hat{e}(C_n) \rightarrow 0$.

Introduce the **average error rate**

$$e(C_n) = \frac{1}{|C_n|} \sum_{c \in C_n} \mathbb{P}(\text{error} \mid c \text{ sent}) \leq \hat{e}(C_n),$$

so $e(C_n) \rightarrow 0$ as well. Let X be uniform on C_n , let Y be the channel output, and let $\hat{X}$ be the decoded codeword; write $p = e(C_n) = \mathbb{P}(X \neq \hat{X})$.

Since X is uniform, $H(X) = \log|C_n| = \log 2^{nR} \geq nR - 1$ for n large. By Fano's inequality applied to X and $\hat{X}$, and then the fact that $\hat{X}$ is a function of Y (so conditioning on Y can only be more informative),

$$H(X \mid Y) \leq H(X \mid \hat{X}) \leq H(p) + p \log(|C_n| - 1) \leq 1 + pnR.$$

Therefore, using Lemma 4.26 for the n th extension,

$$nC \geq I(X; Y) = H(X) - H(X \mid Y) \geq (nR - 1) - (1 + pnR),$$

so that

$$pnR \geq n(R - C) - 2, \quad \text{that is} \quad p \geq \frac{n(R - C) - 2}{nR} \rightarrow \frac{R - C}{R} > 0.$$

This contradicts $p = e(C_n) \rightarrow 0$. Hence no rate above C is achievable. $\square$

The direct half is the celebrated one, and Shannon's proof is a masterpiece of the probabilistic method: rather than constructing a good code, we pick one at random and show that the average is good enough.

Proposition 4.28

Let a BSC have error probability p , and let $R < 1 - H(p)$. Then there are codes C_n of length n and size $|C_n| = 2^{nR}$ with $\rho(C_n) \rightarrow R$ and $e(C_n) \rightarrow 0$.

Note that this concerns the *average* error rate e ; we shall upgrade to $\hat{e}$ afterwards.

Proof. Without loss of generality $p < \frac{1}{2}$. Choose $\varepsilon > 0$ small enough that $p + \varepsilon < \frac{1}{2}$ and $R < 1 - H(p + \varepsilon)$; this is possible by continuity of H . Put $m = \lfloor 2^{n(R + \varepsilon)} \rfloor$ and $r = \lfloor n(p + \varepsilon) \rfloor$. We use minimum distance decoding, breaking ties arbitrarily.

Let $\mathcal{C}$ be the set of all $[n, m]$ -codes, so $|\mathcal{C}| = \binom{2^n}{m}$, and choose $C = \{c_1, \dots, c_m\}$ from

$\mathcal{C}$ uniformly at random. Choose $i \in \{1, \dots, m\}$ uniformly at random too, send c_i , and let Y be the output. Then

$$\mathbb{P}(Y \text{ not decoded as } c_i) = \frac{1}{|\mathcal{C}|} \sum_{C \in \mathcal{C}} e(C),$$

so there exists some $C_n \in \mathcal{C}$ with $e(C_n)$ at most this average. It therefore suffices to show that the left hand side tends to 0.

If $B(Y, r) \cap C = \{c_i\}$ then Y is certainly decoded as c_i , so

$$\mathbb{P}(Y \text{ not decoded as } c_i) \leq \mathbb{P}(c_i \notin B(Y, r)) + \mathbb{P}(B(Y, r) \cap C \supsetneq \{c_i\}),$$

and we handle the two terms separately.

The first term is $\mathbb{P}(d(c_i, Y) > r)$, which is the probability that the channel makes more than $n(p + \varepsilon)$ errors; this tends to 0 by Lemma 4.14.

For the second term, condition on Y and on c_i . Given that $c_i \in B(Y, r)$, each of the other $m - 1$ codewords is uniform over the remaining $2^n - 1$ words, so

$$\mathbb{P}(c_j \in B(Y, r) \mid c_i \in B(Y, r)) = \frac{V(n, r) - 1}{2^n - 1} \leq \frac{V(n, r)}{2^n}$$

for $j \neq i$. Hence, by a union bound and Proposition 3.25(i),

$$\begin{aligned} \mathbb{P}(B(Y, r) \cap C \supsetneq \{c_i\}) &\leq \sum_{j \neq i} \mathbb{P}(c_j \in B(Y, r) \mid c_i \in B(Y, r)) \\ &\leq (m - 1) \frac{V(n, r)}{2^n} \leq 2^{nR} 2^{nH(p+\varepsilon)} 2^{-n} \\ &= 2^{-n(1-H(p+\varepsilon)-R)} \rightarrow 0, \end{aligned}$$

because $R < 1 - H(p + \varepsilon)$. This completes the proof. $\square$

Proposition 4.29

In Proposition 4.28 we may replace e by $\hat{e}$.

Proof. Choose R' with $R < R' < 1 - H(p)$ and apply the previous proposition to R' , obtaining codes C'_n of size $\lfloor 2^{nR'} \rfloor$ with $e(C'_n) \rightarrow 0$. Order the codewords of C'_n by their individual error probabilities and delete the worse half, giving a code C''_n . If more than half the codewords had error probability exceeding $2e(C'_n)$ the average would exceed $e(C'_n)$; so $\hat{e}(C''_n) \leq 2e(C'_n) \rightarrow 0$. Now C''_n has size $\frac{1}{2} \lfloor 2^{nR'} \rfloor \geq 2^{nR}$ for large n , since $2^{nR'-1} = 2^{n(R'-1/n)} \geq 2^{nR}$ eventually. Passing to a subcode of size exactly $\lfloor 2^{nR} \rfloor$ can only decrease $\hat{e}$, and gives $\rho \rightarrow R$. $\square$

Putting the two halves together: the binary symmetric channel with error probability p has operational capacity exactly $1 - H(p)$, which by Example 4.25 is also its information capacity. This is Theorem 4.24 for the BSC.

Remark. The proof is purely existential. It tells us that almost every code of the right size is good, but it gives no way of writing one down, and no way of decoding one efficiently even if we could – minimum distance decoding over 2^{nR} codewords is

hopeless. Finding explicit families of codes approaching capacity, with practical decoding algorithms, took another fifty years. The rest of these notes on coding theory is about algebraic constructions which are far from optimal but eminently practical.

§4.8 An Aside: the Kelly Criterion

Entropy turns up in places that have nothing obviously to do with communication. Here is one, included because it is charming and because it was lectured.

Suppose a biased coin with $\mathbb{P}(\text{head}) = p$ is tossed repeatedly, and that before each toss we may stake money on heads at odds u : a stake of k returns ku if the coin comes up heads and nothing otherwise. Start with a bankroll $X_0 = 1$ and bet a fixed fraction $w \in [0, 1)$ of the current bankroll each time, so

$$X_{n+1} = \begin{cases} X_n(wu + (1 - w)) & \text{if toss } n + 1 \text{ is a head,} \\ X_n(1 - w) & \text{otherwise.} \end{cases}$$

Then $Y_{n+1} = X_{n+1}/X_n$ are i.i.d., and $\log X_n = \sum_{i=1}^n \log Y_i$ is a sum of i.i.d. terms.

Lemma 4.30

Let $\mu = \mathbb{E}[\log Y_1]$ and $\sigma^2 = \text{Var}(\log Y_1)$. Then for $a > 0$,

$$\mathbb{P}\left(\left|\frac{\log X_n}{n} - \mu\right| \geq a\right) = \mathbb{P}\left(\left|\frac{1}{n} \sum_{i=1}^n \log Y_i - \mu\right| \geq a\right) \leq \frac{\sigma^2}{na^2}.$$

In particular, for any $\varepsilon, \delta > 0$ there is N with $\mathbb{P}(|\frac{\log X_n}{n} - \mu| \geq \delta) \leq \varepsilon$ for all $n \geq N$.

Proof. Immediate from Chebyshev's inequality applied to the sample mean. $\square$

So the bankroll behaves like $2^{n\mu}$, and to do well in the long run we should maximise μ rather than the expected return. Now

$$\mu = \mathbb{E}[\log Y] = p \log(1 + (u - 1)w) + (1 - p) \log(1 - w).$$

Differentiating with respect to w and setting the result to zero, one finds that μ is maximised at $w = 0$ if $up \leq 1$ (the bet is not worth taking) and at

$$w = \frac{up - 1}{u - 1}$$

if $up > 1$. This is **Kelly's criterion**. Writing $q = 1 - p$ and substituting the optimal w ,

$$\mathbb{E}[\log Y] = p \log p + q \log q + \log u - q \log(u - 1) = -H(p) + \log u - q \log(u - 1).$$

Entropy has appeared, and it is exactly the penalty paid for the randomness of the coin. Betting below the Kelly fraction still grows the bankroll, but more slowly; betting sufficiently far above it makes the bankroll tend to zero almost surely, however favourable the odds.

§5 Linear Codes

Shannon's theorem guarantees good codes but produces none, and even if it did we could not use them, because minimum distance decoding over an unstructured code of size 2^{nR} is computationally hopeless. The way out is to insist that our codes be *linear subspaces*, at which point all the machinery of IB Linear Algebra becomes available and both encoding and decoding become matrix arithmetic.

§5.1 Linear Codes, Generator and Parity Check Matrices

Definition 5.1 (Linear Code)

A binary code $C \subseteq \mathbb{F}_2^n$ is **linear** if $0 \in C$ and $x + y \in C$ whenever $x, y \in C$; equivalently, if C is a vector subspace of $\mathbb{F}_2^n$.

Definition 5.2 (Rank)

The **rank** of a linear code C is its dimension as an $\mathbb{F}_2$ -vector space. A linear code of length n and rank k is an **(n, k) -code**; if its minimum distance is d it is an **(n, k, d) -code**.

We use round brackets for linear codes and square brackets for general ones. If $v_1, \dots, v_k$ is a basis then every codeword is uniquely $\sum \lambda_i v_i$ with $\lambda_i \in \mathbb{F}_2$, so $|C| = 2^k$. Thus an (n, k) -code is an $[n, 2^k]$ -code, and its information rate is $\frac{k}{n}$ – a pleasantly simple formula.

Definition 5.3 (Weight)

The **weight** of $x \in \mathbb{F}_2^n$ is $w(x) = d(x, 0)$, the number of nonzero digits.

Lemma 5.4

The minimum distance of a linear code equals the minimum weight of a nonzero codeword.

Proof. For $x, y \in \mathbb{F}_2^n$ we have $d(x, y) = d(x + y, 0) = w(x + y)$, since over $\mathbb{F}_2$ subtraction is addition. If C is linear and $x, y \in C$ are distinct then $x + y$ is a nonzero codeword, and conversely every nonzero codeword z arises as $z = z + 0$. So the two minima are over the same set of numbers. $\square$

This is already a large saving: computing $d(C)$ naively requires $\binom{2^k}{2}$ comparisons, but for a linear code only $2^k - 1$ weights.

Definition 5.5 (Dot Product, Dual Code)

For $x, y \in \mathbb{F}_2^n$ set $x \cdot y = \sum_{i=1}^n x_i y_i \in \mathbb{F}_2$; this is symmetric and bilinear. Given $P \subseteq \mathbb{F}_2^n$, the **parity check code** defined by P is

$$C = \{x \in \mathbb{F}_2^n : p \cdot x = 0 \text{ for all } p \in P\}.$$

For a linear code C , the **dual code** is

$$C^\perp = \{x \in \mathbb{F}_2^n : x \cdot y = 0 \text{ for all } y \in C\}.$$

A warning: this bilinear form is not an inner product, since $x \cdot x = 0$ for any x of even weight. In particular $C \cap C^\perp$ need not be $\{0\}$, and a code can even be *self-dual*.

Example 5.6 (i) $P = \{11 \dots 1\}$ gives the simple parity check code.

(ii) $P = \{1010101, 0110011, 0001111\}$ gives Hamming's original $[7, 16, 3]$ -code of Example 3.8.

(iii) If C is linear then so are the parity check extension C^+ and any punctured code C^- .

Lemma 5.7

Every parity check code is linear.

Proof. $p \cdot 0 = 0$, and if $p \cdot x = p \cdot y = 0$ then $p \cdot (x + y) = 0$ by bilinearity. $\square$

In particular $C^\perp$ is always linear. We shall shortly see the converse: every linear code is a parity check code.

Definition 5.8 (Generator and Parity Check Matrices)

Let C be an (n, k) -code. A **generator matrix** for C is a $k \times n$ matrix G whose rows form a basis of C . A **parity check matrix** for C is a generator matrix H for $C^\perp$; we shall see it is an $(n - k) \times n$ matrix.

The two matrices give two descriptions of the same code: codewords are the linear combinations of rows of G , and also the linear dependence relations among the columns of H ,

$$C = \{x \in \mathbb{F}_2^n : Hx = 0\},$$

where we regard x as a column vector. Encoding uses G and decoding uses H .

Definition 5.9 (Syndrome)

Let C be a linear code with parity check matrix H . The **syndrome** of $x \in \mathbb{F}_2^n$ is $Hx \in \mathbb{F}_2^{n-k}$.

Here is the point. Suppose we receive $x = c + z$ where $c \in C$ and z is the error pattern. Since $Hc = 0$,

$$Hx = Hz,$$

so the syndrome depends only on the error, not on the codeword that was sent. If C is e -error correcting we may precompute Hx for every z with $w(z) \leq e$ – there are $V(n, e)$ of them, far fewer than 2^k codewords in general – and store the results in a table. On receiving x we compute Hx , look it up, and decode as $c = x - z$. The cost of decoding is one matrix multiplication and one table lookup.

More generally, two words have the same syndrome exactly when they differ by a code-word, so the syndromes are in bijection with the cosets of C in $\mathbb{F}_2^n$. Minimum distance decoding amounts to choosing, in each coset, a word of least weight – a **coset leader** – and subtracting it. Displaying the cosets as the rows of an array gives the classical picture.

z	coset $z + C$				$H z$
0	0	c_2	$\cdots$	c_m	0
z_2	z_2	$z_2 + c_2$	$\cdots$	$z_2 + c_m$	s_2
z_3	z_3	$z_3 + c_2$	$\cdots$	$z_3 + c_m$	s_3
$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$
z_t	z_t	$z_t + c_2$	$\cdots$	$z_t + c_m$	s_t

The first row is the code itself; the leftmost column holds the coset leaders; the rightmost column holds the syndromes, and it is the only column we need to store. On receiving x we find the row with syndrome Hx and subtract its leader.

Definition 5.10 (Equivalence)

Codes $C_1, C_2 \subseteq \mathbb{F}_2^n$ are **equivalent** if some permutation of the n coordinates carries C_1 onto C_2 .

Equivalent codes have the same length, size and minimum distance, so we generally work up to equivalence.

Lemma 5.11

Every (n, k) -code is equivalent to one with generator matrix in the **standard form**

$$G = (I_k \mid B)$$

for some $k \times (n - k)$ matrix B .

Proof. Let G be any generator matrix. Row operations do not change the row space, so by Gaussian elimination we may put G in row echelon form, with leading 1s in columns $\ell(1) < \ell(2) < \cdots < \ell(k)$. Permuting columns replaces C by an equivalent code, so we may assume $\ell(i) = i$. Further row operations clear the entries above the leading 1s, leaving $G = (I_k \mid B)$. $\square$

With G in standard form, a message $y \in \mathbb{F}_2^k$ (a row vector) is encoded as

$$yG = (y \mid yB),$$

that is, the message itself followed by $n - k$ **check digits**. Such an encoding is called **systematic**, and is convenient because the message can be read straight off the code-word.

Lemma 5.12

For a linear code $C \leq \mathbb{F}_2^n$, $\text{rank } C + \text{rank } C^\perp = n$.

Proof. This is the annihilator statement from IB Linear Algebra, but here is a direct argument. Passing to an equivalent code we may assume C has generator matrix $G = (I_k \mid B)$. Consider the linear map $\gamma: \mathbb{F}_2^n \rightarrow \mathbb{F}_2^k$, $x \mapsto Gx$. Since G contains I_k , its rows are linearly independent, so γ is surjective. Its kernel is exactly the set of x orthogonal to every row of G , that is, $C^\perp$. By rank-nullity,

$$n = \dim \ker \gamma + \dim \text{img } \gamma = \text{rank } C^\perp + k. \quad \square$$

Corollary 5.13

$(C^\perp)^\perp = C$. In particular every linear code is a parity check code, namely the one defined by $C^\perp$.

Proof. If $x \in C$ then $x \cdot y = 0$ for all $y \in C^\perp$ by definition, so $C \subseteq (C^\perp)^\perp$. Applying Lemma ?? twice,

$$\text{rank}(C^\perp)^\perp = n - \text{rank } C^\perp = n - (n - \text{rank } C) = \text{rank } C,$$

so the inclusion is an equality. $\square$

Lemma 5.14

If C has generator matrix $G = (I_k \mid B)$ then $H = (B^\top \mid I_{n-k})$ is a parity check matrix for C .

Proof. We compute

$$GH^\top = (I_k \mid B) \begin{pmatrix} B \\ I_{n-k} \end{pmatrix} = B + B = 0$$

over $\mathbb{F}_2$. So every row of H is orthogonal to every row of G , giving $\text{rank } H \leq \dim C^\perp$ as subspaces; but H contains I_{n-k} , so $\text{rank } H = n - k = \text{rank } C^\perp$ by Lemma ??. Hence the rows of H are a basis of $C^\perp$. $\square$

The minimum distance can also be read off from H , which is often the most convenient way to compute it.

Lemma 5.15

Let C be a linear code with parity check matrix H . Then $d(C) = d$ if and only if some d columns of H are linearly dependent, but no $d - 1$ columns are.

Proof. A word x lies in C if and only if $Hx = 0$, that is, if and only if the columns of H indexed by the support of x sum to zero. So there is a nonzero codeword of weight w exactly when some w columns of H are linearly dependent. Now apply Lemma ??. $\square$

Example 5.16

Let C be the $(5, 2)$ -code with generator matrix

$$G = \begin{pmatrix} 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{pmatrix} = (I_2 \ B), \quad B = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}.$$

Its four codewords are 00000, 10110, 01011, 11101, of weights 0, 3, 3, 4, so $d(C) = 3$ by Lemma ?? and C is a $(5, 2, 3)$ -code, correcting one error. By Lemma ??,

$$H = (B^\top \ I_3) = \begin{pmatrix} 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \end{pmatrix},$$

and indeed no two columns of H are equal (so no two are dependent), while columns 1, 2, 4 sum to zero, confirming $d = 3$ via Lemma ??.

The syndromes of the five single-bit errors are just the columns of H :

z	00000	10000	01000	00100	00010	00001	00101	10001
$H z$	000	110	011	100	010	001	101	111

There are 8 syndromes and only 6 patterns of weight at most 1, so two cosets need leaders of weight 2; the last two columns give such a choice. Note the choice is not unique – the syndrome 101 also arises from 11000, and 111 from 01100 – which is Lemma 3.11 telling us that a code with $d = 3$ cannot correct two errors.

Let us decode. Suppose we receive $x = 10111$. Then

$$Hx = (1 + 1, 1 + 1, 1 + 1)^\top \cdot (\text{reading off columns 1, 3, 4, 5}) = 001,$$

more carefully: the columns of H in positions 1, 3, 4, 5 are $(1, 1, 0)$, $(1, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$, which sum to $(0, 0, 1)$. Looking up 001 in the table gives $z = 00001$, so we decode x as $10111 + 00001 = 10110$, a codeword as required.

§5.2 Hamming Codes

Lemma ?? makes the construction of 1-error correcting codes almost automatic. We need $d = 3$, so we need a parity check matrix whose columns are pairwise linearly independent; over $\mathbb{F}_2$ that just means the columns are nonzero and distinct. To get as many columns as possible for a given number of rows, we should use *all* of them.

Definition 5.17 (Hamming Code)

Let $d \geq 2$ and $n = 2^d - 1$. Let H be the $d \times n$ matrix whose columns are the n nonzero elements of $\mathbb{F}_2^d$, in some order. The **Hamming $(n, n - d)$ -code** is the linear code with parity check matrix H .

Lemma 5.18

The Hamming $(n, n - d)$ -code has minimum distance 3 and is a perfect 1-error correcting code.

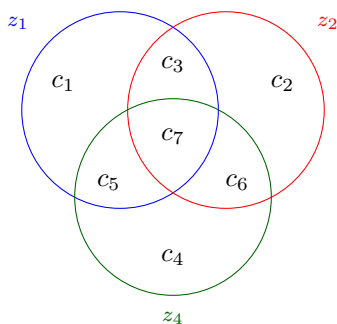
Proof. The columns of H are distinct and nonzero, so no two are dependent; and any three columns $u, v, u + v$ are dependent, and such a triple exists as soon as $d \geq 2$. By Lemma ??, $d(C) = 3$, so C corrects $\lfloor \frac{3-1}{2} \rfloor = 1$ error. Finally

$$\frac{2^n}{V(n, 1)} = \frac{2^n}{1 + n} = \frac{2^n}{2^d} = 2^{n-d} = |C|,$$

so C is perfect. □

If the columns of H are listed in the order $1, 2, \dots, n$ read as binary numbers, then the syndrome of a single error in position i is the binary expansion of i , and decoding is instantaneous: compute the syndrome, read it as a number, and flip that bit. Taking $d = 3$ recovers Hamming's original $(7, 4, 3)$ -code of Example 3.8.

That code has a famous picture. Draw three overlapping circles, one for each parity check, and place the seven digits in the seven regions so that digit i lies in circle j exactly when the j th binary digit of i is 1.



Each circle must contain an even number of 1s: those are exactly the three parity conditions

$$c_1 + c_3 + c_5 + c_7 = 0, \quad c_2 + c_3 + c_6 + c_7 = 0, \quad c_4 + c_5 + c_6 + c_7 = 0.$$

Now the decoding rule becomes visual. A single error breaks precisely the parity of those circles containing it, and since every region is determined by the set of circles containing it, the broken circles identify the erroneous digit uniquely. If, for instance, only the blue and green circles fail, the culprit is c_5 , which lies in those two and no other.

§5.3 Weight Enumerators

The weights of the codewords of a linear code contain a great deal of information, and it is convenient to package them into a polynomial.

Definition 5.19 (Weight Enumerator)

The **weight enumerator** of a linear code $C \leq \mathbb{F}_2^n$ is the homogeneous polynomial

$$W_C(x, y) = \sum_{c \in C} x^{n-w(c)} y^{w(c)} = \sum_{i=0}^n A_i x^{n-i} y^i,$$

where A_i is the number of codewords of weight i .

Note $W_C(1, 1) = |C|$ and $W_C(1, 0) = 1$, and that $d(C)$ is the least $i > 0$ with $A_i \neq 0$. If C is used over a BSC with error probability p , then the probability that a transmitted codeword is received as another codeword – an undetected error – is $W_C(1-p, p) - (1-p)^n$, so the weight enumerator is exactly what one needs to analyse error detection.

Example 5.20

The repetition code of length n has $W(x, y) = x^n + y^n$. The simple parity check code of length n has $W(x, y) = \sum_{i \text{ even}} \binom{n}{i} x^{n-i} y^i = \frac{1}{2}[(x+y)^n + (x-y)^n]$. Hamming's $(7, 4, 3)$ -code has weight distribution $A_0 = 1$, $A_3 = A_4 = 7$, $A_7 = 1$, so

$$W(x, y) = x^7 + 7x^4y^3 + 7x^3y^4 + y^7.$$

(One checks $A_3 = 7$ directly: a weight-3 codeword corresponds to a triple of columns of H summing to zero, and each of the 7 nonzero syndromes u determines such a triple. The remaining counts follow since C contains 1111111, so $A_i = A_{7-i}$.)

There is a beautiful and useful relation between the weight enumerator of a code and that of its dual, which comes from Fourier analysis on $\mathbb{F}_2^n$.

Lemma 5.21 (Poisson Summation)

Let $C \leq \mathbb{F}_2^n$ be linear and let $f: \mathbb{F}_2^n \rightarrow \mathbb{R}$, with Fourier transform $\hat{f}(v) = \sum_{u \in \mathbb{F}_2^n} (-1)^{u \cdot v} f(u)$. Then

$$\sum_{v \in C} \hat{f}(v) = |C| \sum_{u \in C^\perp} f(u).$$

Proof. Exchanging the order of summation,

$$\sum_{v \in C} \hat{f}(v) = \sum_{v \in C} \sum_u (-1)^{u \cdot v} f(u) = \sum_u f(u) \sum_{v \in C} (-1)^{u \cdot v}.$$

The inner sum is $|C|$ if $u \in C^\perp$. Otherwise the map $v \mapsto u \cdot v$ is a surjective linear map $C \rightarrow \mathbb{F}_2$, so its kernel has index 2 in C and the sum is $\frac{1}{2}|C| - \frac{1}{2}|C| = 0$. $\square$

Theorem 5.22 (MacWilliams Identity)

For a linear code $C \leq \mathbb{F}_2^n$,

$$W_{C^\perp}(x, y) = \frac{1}{|C|} W_C(x+y, x-y).$$

Proof. Apply Lemma ?? to $f(u) = x^{n-w(u)} y^{w(u)}$, which factorises as $f(u) = \prod_{i=1}^n g(u_i)$ with $g(0) = x$, $g(1) = y$. Then the Fourier transform factorises too:

$$\hat{f}(v) = \sum_{u \in \mathbb{F}_2^n} \prod_{i=1}^n (-1)^{u_i v_i} g(u_i) = \prod_{i=1}^n \left(\sum_{u_i \in \mathbb{F}_2} (-1)^{u_i v_i} g(u_i) \right) = \prod_{i=1}^n (x + (-1)^{v_i} y),$$

which is $(x+y)^{n-w(v)} (x-y)^{w(v)}$. Summing over $v \in C$ gives $W_C(x+y, x-y)$ on the left of Lemma ??, and $|C| W_{C^\perp}(x, y)$ on the right. $\square$

Example 5.23

Take C the simple parity check code of length n , with $W_C(x, y) = \frac{1}{2}[(x+y)^n + (x-y)^n]$ and $|C| = 2^{n-1}$. Then

$$W_{C^\perp}(x, y) = \frac{1}{2^{n-1}} \cdot \frac{1}{2}[(2x)^n + (2y)^n] = x^n + y^n,$$

which is the weight enumerator of the repetition code. And indeed the dual of the parity check code is the repetition code, as one sees directly.

§5.4 Reed–Muller Codes

Our final family of linear codes is built out of the geometry of $\mathbb{F}_2^d$, and includes several codes we have already met. It is the family used by NASA to transmit the Mariner photographs of Mars.

Let $X = \{p_1, \dots, p_n\}$ be a set of size n . There is a chain of bijections

$$\mathcal{P}(X) \xrightarrow{A \mapsto \mathbb{1}_A} \{f: X \rightarrow \mathbb{F}_2\} \xrightarrow{f \mapsto (f(p_1), \dots, f(p_n))} \mathbb{F}_2^n,$$

under which the symmetric difference $A \triangle B$ corresponds to vector addition, and the intersection $A \cap B$ corresponds to the **wedge product**

$$x \wedge y = (x_1 y_1, \dots, x_n y_n).$$

So we may think of vectors in $\mathbb{F}_2^n$ as subsets of X , and we shall pass between the two pictures freely.

Now take $X = \mathbb{F}_2^d$, so that $n = |X| = 2^d$. Let $v_0 = (1, 1, \dots, 1) = \mathbb{1}_X$, and for $1 \leq i \leq d$ let

$$v_i = \mathbb{1}_{H_i}, \quad H_i = \{p \in X : p_i = 0\},$$

the indicator of the i th coordinate hyperplane.

Definition 5.24 (Reed–Muller Code)

For $0 \leq r \leq d$, the **Reed–Muller code** $RM(d, r)$ of **order** r and length 2^d is the linear code spanned by v_0 together with all wedge products of at most r of the v_i , $1 \leq i \leq d$. (By convention the empty wedge product is v_0 .)

Example 5.25

Let $d = 3$ and list $X = \mathbb{F}_2^3$ in binary order. Then the spanning vectors are

X	000	001	010	011	100	101	110	111
v_0	1	1	1	1	1	1	1	1
v_1	1	1	1	1	0	0	0	0
v_2	1	1	0	0	1	1	0	0
v_3	1	0	1	0	1	0	1	0
$v_1 \wedge v_2$	1	1	0	0	0	0	0	0
$v_1 \wedge v_3$	1	0	1	0	0	0	0	0
$v_2 \wedge v_3$	1	0	0	0	1	0	0	0
$v_1 \wedge v_2 \wedge v_3$	1	0	0	0	0	0	0	0

Notice that deleting the first column and the rows v_0 and $v_1 \wedge v_2 \wedge v_3$ leaves a generator matrix for a $(7, 4)$ -code, which turns out to be Hamming's original code.

We can identify the small cases directly. $RM(3, 0)$ is spanned by v_0 alone, so it is the repetition code of length 8. $RM(3, 1)$ is spanned by v_0, v_1, v_2, v_3 , and is equivalent to the parity check extension of Hamming's $(7, 4)$ -code. $RM(3, 2)$ is an $(8, 7)$ -code, equivalent to the simple parity check code of length 8. And $RM(3, 3)$ is the whole of $\mathbb{F}_2^8$.

Theorem 5.26 (i) The 2^d vectors $v_{i_1} \wedge \cdots \wedge v_{i_s}$ with $i_1 < \cdots < i_s$ and $0 \leq s \leq d$ form a basis of $\mathbb{F}_2^n$, where $n = 2^d$.
(ii) $\text{rank } RM(d, r) = \sum_{s=0}^r \binom{d}{s}$.

Proof. (i). There are $\sum_{s=0}^d \binom{d}{s} = 2^d = n$ vectors in the list, so it suffices to show they span, that is, that $RM(d, d) = \mathbb{F}_2^n$. Fix $p \in X$ and set

$$y_i = \begin{cases} v_i & \text{if } p_i = 0, \\ v_0 + v_i & \text{if } p_i = 1. \end{cases}$$

In the set picture y_i is the indicator of $\{q : q_i = p_i\}$, so

$$y_1 \wedge \cdots \wedge y_d = \mathbb{1}_{\{p\}}.$$

Expanding the left hand side by distributivity of $\wedge$ over $+$ writes $\mathbb{1}_{\{p\}}$ as a sum of wedge products of the v_i , hence $\mathbb{1}_{\{p\}} \in RM(d, d)$. As p was arbitrary and the $\mathbb{1}_{\{p\}}$ span $\mathbb{F}_2^n$, we are done.

(ii). $RM(d, r)$ is spanned by those basis vectors with $s \leq r$, of which there are $\sum_{s \leq r} \binom{d}{s}$, and these are linearly independent by (i). $\square$

To compute the minimum distance we use a construction which is of independent interest.

Definition 5.27 (Bar Product)

Let $C_2 \subseteq C_1$ be linear codes of length n . Their **bar product** is

$$C_1 \mid C_2 = \{(x \mid x + y) : x \in C_1, y \in C_2\},$$

a linear code of length $2n$.

Lemma 5.28 (i) $\text{rank}(C_1 \mid C_2) = \text{rank } C_1 + \text{rank } C_2$;

(ii) $d(C_1 \mid C_2) = \min\{2d(C_1), d(C_2)\}$.

Proof. (i). If $x_1, \dots, x_k$ is a basis of C_1 and $y_1, \dots, y_\ell$ a basis of C_2 , then

$$\{(x_i \mid x_i) : i \leq k\} \cup \{(0 \mid y_j) : j \leq \ell\}$$

spans $C_1 \mid C_2$, and is independent: a vanishing combination must have zero first

half, forcing all x -coefficients to vanish, and then the y -coefficients too.

(ii). Let $(x \mid x+y)$ be a nonzero element. If $y \neq 0$ then, since weight is subadditive and $w(u) + w(u+v) \geq w(v)$,

$$w(x \mid x+y) = w(x) + w(x+y) \geq w(y) \geq d(C_2).$$

If $y = 0$ then $x \neq 0$ and $w(x \mid x) = 2w(x) \geq 2d(C_1)$. Hence $d(C_1 \mid C_2) \geq \min\{2d(C_1), d(C_2)\}$. For the reverse, choose $x \in C_1$ of weight $d(C_1)$ to get a codeword $(x \mid x)$ of weight $2d(C_1)$, and choose $y \in C_2$ of weight $d(C_2)$ to get $(0 \mid y)$ of weight $d(C_2)$. $\square$

Theorem 5.29 (i) $RM(d, r) = RM(d-1, r) \mid RM(d-1, r-1)$ for $0 < r < d$;
(ii) $RM(d, r)$ has minimum distance 2^{d-r} .

Proof. (i). Order $X = \mathbb{F}_2^d$ so that the points with last coordinate 0 come first, identifying the two halves with $\mathbb{F}_2^{d-1}$. Under this splitting, the vectors $v_0, v_1, \dots, v_{d-1}$ for $\mathbb{F}_2^d$ become $(w \mid w)$ where w is the corresponding vector for $\mathbb{F}_2^{d-1}$, while $v_d = (\mathbf{1} \mid \mathbf{0})$. A wedge product of at most r of the v_i with $i < d$ therefore has the form $(w \mid w)$ with $w \in RM(d-1, r)$; and a wedge product involving v_d has the form $(w \mid 0)$ with $w \in RM(d-1, r-1)$, since one of its r factors has been used up. As $(w \mid 0) = (w \mid w+w)$, every generator lies in $RM(d-1, r) \mid RM(d-1, r-1)$; and comparing ranks using Theorem ??(ii), Lemma ??(i) and Pascal's rule

$$\sum_{s \leq r} \binom{d}{s} = \sum_{s \leq r} \binom{d-1}{s} + \sum_{s \leq r-1} \binom{d-1}{s}$$

shows the two codes have the same dimension, hence are equal.

(ii). If $r = 0$ then $RM(d, 0)$ is the repetition code of length 2^d , of minimum distance 2^d . If $r = d$ then $RM(d, d) = \mathbb{F}_2^{2^d}$, of minimum distance $1 = 2^{d-d}$. For $0 < r < d$ we induct on d , using (i) and Lemma ??(ii):

$$d(RM(d, r)) = \min\{2 \cdot 2^{(d-1)-r}, 2^{(d-1)-(r-1)}\} = \min\{2^{d-r}, 2^{d-r}\} = 2^{d-r}. \quad \square$$

Example 5.30

$RM(5, 1)$ is a $(32, 6)$ -code with minimum distance $2^4 = 16$, so it corrects 7 errors at rate $\frac{6}{32}$. This is the code used by Mariner 9 in 1971 to send back pictures of Mars: each pixel was one of 64 grey levels, encoded in 32 bits, and the channel was extremely noisy.